NumPy, Measured

Arrays, dtypes, views, broadcasting and the mistakes that give no error

8 chapters · 14,227 words

Every number is reproduced by code that ships with the book.

Contents

Chapters

Chapter 1 — The array 3

Chapter 2 — Dtypes, and the answers that change silently 7

Chapter 3 — Views and copies 12

Chapter 4 — Indexing 18

Chapter 5 — Broadcasting 24

Chapter 6 — Ufuncs and vectorization 32

Chapter 7 — Linear algebra and random numbers 40

Chapter 8 — Putting it together 46

Exercises

Exercises — Chapter 1 52

Exercises — Chapter 2 53

Exercises — Chapter 3 54

Exercises — Chapter 4 55

Exercises — Chapter 5 56

Exercises — Chapter 6 57

Exercises — Chapter 7 58

Exercises — Chapter 8 59

Worked solutions

Solutions — Chapter 1 60

Solutions — Chapter 2 62

Solutions — Chapter 3 64

Solutions — Chapter 4 66

Solutions — Chapter 5 68

Solutions — Chapter 6 70

Solutions — Chapter 7 72

Solutions — Chapter 8 74

Chapter 1 — The array

> **Build** — a function `describe(a)` that prints the memory layout of any array, and a measurement of how much faster an array sum is than a list sum.
>
> **Theory kept** — an array is one block of memory plus three numbers per axis: the shape, the dtype and the strides. Nothing else.
>
> **Source** — New. The behavior is from the NumPy documentation for `numpy.ndarray`. Every number is from this book's code.
>
> **Run this first** — `uv run python -m examples.ch01_array`, which is `examples/ch01_array.py`.

A NumPy array is a block of memory and a description of how to read it. The block holds raw values, side by side, with no gaps. The description tells NumPy where one row ends and the next row starts. When you know these two facts, most of NumPy is no surprise.

This chapter shows the description, measures what it buys, and shows one case where it buys nothing.

The description: shape, dtype, strides

An array has three attributes that fix its layout.

- `shape` is a tuple with the length of each axis.
- `dtype` is the type of each value. It fixes the size of one value in bytes, which is `itemsize`.
- `strides` is a tuple with the number of bytes to jump to move one step along each axis.

Make a 3-by-4 array of 32-bit integers and print the three attributes.

```
import numpy as np

a = np.arange(12, dtype=np.int32).reshape(3, 4)
print(a)
print("shape   ", a.shape)
print("itemsize", a.itemsize)
print("strides ", a.strides)
print("nbytes  ", a.nbytes)
```

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
shape    (3, 4)
itemsize 4
strides  (16, 4)
nbytes   48
```

Read the strides as a rule. To move one column to the right, jump 4 bytes, which is one `int32`. To move one row down, jump 16 bytes, which is 4 values of 4 bytes. The array has 12 values of 4 bytes each, so `nbytes` is 48.

The address of the value at row `i` and column `j` is therefore the start of the block plus `i * 16 + j * 4`. There is no other rule. A slice, a transpose and a reshape all work the same way. They change these numbers and do not touch the block.

A transpose copies nothing

Transpose the array and print its layout again.

```
import numpy as np

a = np.arange(12, dtype=np.int32).reshape(3, 4)
b = a.T
print("a strides", a.strides, "shape", a.shape)
print("b strides", b.strides, "shape", b.shape)
print("same memory:", np.shares_memory(a, b))
print("b is C-contiguous:", b.flags.c_contiguous)
```

```
a strides (16, 4) shape (3, 4)
b strides (4, 16) shape (4, 3)
same memory: True
b is C-contiguous: False
```

The shape went from `(3, 4)` to `(4, 3)`, and the strides swapped from `(16, 4)` to `(4, 16)`. The 48 bytes did not move. `np.shares_memory` confirms that `a` and `b` read the same block.

The last line shows the price. A C-contiguous array has its last axis in consecutive memory, so a walk along the last axis reads memory in order. The transpose does not have this property. A loop over its last axis jumps 16 bytes at each step. Chapter 3 measures what that costs.

The function you build

Put the three attributes and the two flags that matter into one function. You will use it in every later chapter.

```
import numpy as np


def describe(a: np.ndarray) -> str:
    """One line that tells the whole layout of an array."""
    return (f"shape={a.shape} dtype={a.dtype} itemsize={a.itemsize} "
            f"strides={a.strides} nbytes={a.nbytes} "
            f"c_contiguous={a.flags.c_contiguous} owns={a.flags.owndata}")


a = np.arange(12, dtype=np.int32).reshape(3, 4)
print(describe(a))
print(describe(a.T))
print(describe(a[:, ::2]))
```

```
shape=(3, 4) dtype=int32 itemsize=4 strides=(16, 4) nbytes=48 c_contiguous=True owns=False
shape=(4, 3) dtype=int32 itemsize=4 strides=(4, 16) nbytes=48 c_contiguous=False owns=False
shape=(3, 2) dtype=int32 itemsize=4 strides=(16, 8) nbytes=24 c_contiguous=False owns=False
```

The `owns` field is `True` when the array owns its block. It is `False` when the array only looks at the block of another array. The slice `a[:, ::2]` takes every second column. It has strides `(16, 8)`, so it reads the same block with a step of 8 bytes.

The first line is a surprise. The array `a` itself has `owns=False`. The call `np.arange(12)` made the block, and `reshape` returned a new view of that block. The variable `a` is a view, and the block belongs to an array that no name points to. Chapter 3 shows why this matters.

Why a sum over an array is fast

A Python list holds pointers. Each pointer leads to a separate integer object somewhere in memory. A sum over the list must follow each pointer, check the type of the object, and add. An array holds the values themselves, so a sum is one tight loop over consecutive bytes.

Measure both. The code below sums one million integers in each form and keeps the best of five runs.

```
import timeit

import numpy as np

n = 1_000_000
as_list = list(range(n))
as_array = np.arange(n)

t_list = min(timeit.repeat(lambda: sum(as_list), number=5, repeat=5)) / 5
t_array = min(timeit.repeat(lambda: as_array.sum(), number=5, repeat=5)) / 5
ratio = t_list / t_array
assert sum(as_list) == int(as_array.sum())
assert ratio > 5, "chapter 1 claims an array sum is at least 5 times faster"
print(f"list sum   {t_list * 1e3:8.3f} ms")
print(f"array sum  {t_array * 1e3:8.3f} ms")
print(f"ratio      {ratio:8.1f}x")
```

```
# output varies
list sum      9.2 ms
array sum     0.7 ms
ratio        13x
```

The numbers above are from one run on the machine that built this book. Yours will differ. The `assert` is the claim: the array sum is at least 5 times faster. The run on the build machine gave a ratio of about 13. The assert fails if the ratio falls below 5 on yours.

Memory shows the same effect. The list holds 8-byte pointers and, in addition, one 28-byte object for each integer above 256.

```
import sys

import numpy as np

n = 1_000_000
as_list = list(range(n))
list_bytes = sys.getsizeof(as_list) + sum(sys.getsizeof(x) for x in as_list)
array_bytes = np.arange(n).nbytes
print("list  ", list_bytes, "bytes")
print("array ", array_bytes, "bytes")
print("ratio ", round(list_bytes / array_bytes, 1))
```

```
list   36000056 bytes
array  8000000 bytes
ratio  4.5
```

The list needs 4.5 times the memory of the array for the same one million integers. The ratio depends on the dtype. An `int8` array of the same length is 36 times smaller than the list.

A case where the array is slower

The array has a fixed cost for each call. NumPy must check the type, allocate a result and start the loop. For a very small input, this cost is larger than the work.

```
import timeit

import numpy as np

small_list = [1, 2, 3]
small_array = np.array(small_list)
t_list = min(timeit.repeat(lambda: sum(small_list), number=20000, repeat=5)) / 20000
t_array = min(timeit.repeat(lambda: small_array.sum(), number=20000, repeat=5)) / 20000
print("array is slower for 3 values:", t_array > t_list)
assert t_array > t_list
```

```
array is slower for 3 values: True
```

The list wins for three values. Use NumPy when you have thousands of values or more, or when the shape of the problem is a table. For a handful of numbers, use a list and do not apologize.

What you have now

You can read any array as a block plus `(shape, dtype, strides)`. You have a `describe` function that prints this. You measured three results. The array sum is about 13 times faster on one machine. The memory is 4.5 times smaller. Three values are faster in a list.

Chapter 2 looks at the `dtype` field, which decides what a value is, and at the ways it silently changes the answer.

Chapter 2 — Dtypes, and the answers that change silently

> **Build** — `checked_add(a, b)`, an integer add that raises an error when a result does not fit the dtype, in place of wrapping around.
>
> **Theory kept** — a dtype fixes how many bits a value has. NumPy does not check that a result fits in those bits. It keeps the bits that fit and drops the rest.
>
> **Source** — New. The promotion rules are NEP 50, which NumPy 2 implements. Every output below is from this book's code.
>
> **Run this first** — `uv run python -m examples.ch02_dtypes`, which is `examples/ch02_dtypes.py`.

Chapter 1 gave every array a dtype. This chapter shows what the dtype does to your numbers. Four things go wrong in practice.

- Integers wrap.
- Floats lose small additions.
- A sum uses a different dtype than its input.
- A mixed expression picks a dtype that you did not choose.

Each one gives a wrong answer and no error.

What NumPy picks when you do not choose

A Python integer becomes `int64` on this machine. A Python float becomes `float64`. A mix of the two becomes `float64`.

```
import numpy as np

print(np.array([1, 2, 3]).dtype)
print(np.array([1.0, 2]).dtype)
print(np.array([1, 2.5]).dtype)
print(np.array([1, 2, 3]).sum().dtype)
print((np.array([1, 2, 3]) / 2).dtype)
```

```
int64
float64
float64
int64
float64
```

The last line is a common surprise. The division operator `/` always gives floats, even for integer input. Use `//` for integer division.

Integers wrap around

An `int8` holds the values −128 to 127. NumPy does not stop you at the edge. It wraps.

```
import numpy as np

a = np.array([100, 120, 127], dtype=np.int8)
print(a + 1)
print((a + 1).dtype)

img = np.array([200, 250, 10], dtype=np.uint8)
print(img + 100)
print(np.array([300, -5]).astype(np.uint8))
```

```
[ 101  121 -128]
int8
[ 44  94 110]
[ 44 251]
```

The value 127 plus 1 is −128. The pixel 200 plus 100 is 44, not 300 and not 255. The call `astype(np.uint8)` turned 300 into 44 and −5 into 251. NumPy printed no message for any of these.

This is the failure that matters most for image data. A brightness boost on a `uint8` image does not clip at 255. It wraps, and the bright areas turn dark.

NumPy 2 does raise one error. A Python integer that does not fit the dtype of the array is an `OverflowError`, as the next block shows.

```
import numpy as np

img = np.array([200, 250, 10], dtype=np.uint8)
try:
    img + 300
except OverflowError as e:
    print(type(e).__name__, "-", e)
```

```
OverflowError - Python integer 300 out of bounds for uint8
```

The error is only for a Python integer that is too large for the dtype. It does not apply when both operands are arrays. The first example, `img + 100`, passed because 100 fits in `uint8`. The result of the add did not fit, and nobody checked.

The function you build

The rule for a safe add is short. Do the add in a wider type. Compare each result with the limits of the narrow type. Raise an error if one result is outside the limits. `np.iinfo` gives the limits.

```
import numpy as np


def checked_add(a: np.ndarray, b: np.ndarray) -> np.ndarray:
    """Add two integer arrays. Raise OverflowError if a result does not fit the dtype of
`a`."""
    if a.dtype.kind not in "iu" or b.dtype.kind not in "iu":
        raise TypeError("checked_add takes integer arrays only")
    if a.dtype == np.uint64 or b.dtype == np.uint64:
        raise TypeError("uint64 has no wider integer type to check against")
    wide = a.astype(np.int64) + b.astype(np.int64)
    info = np.iinfo(a.dtype)
    if wide.size and (wide.min() < info.min or wide.max() > info.max):
        bad = int(np.flatnonzero((wide < info.min) | (wide > info.max))[0])
        raise OverflowError(f"element {bad}: {int(wide.flat[bad])} does not fit {a.dtype}")
    return wide.astype(a.dtype)


pixels = np.array([200, 250, 10], dtype=np.uint8)
try:
    checked_add(pixels, np.array([100, 100, 100], dtype=np.uint8))
except OverflowError as e:
    print("refused:", e)
print(checked_add(pixels, np.array([10, 5, 100], dtype=np.uint8)))
```

```
refused: element 0: 300 does not fit uint8
[210 255 110]
```

The function has two limits, and the code says both. It cannot check `uint64`, because no wider integer type exists in NumPy. It takes `a.dtype` as the output type, so a `uint8` plus an `int16` is checked against `uint8`. For a clip instead of an error, use `np.clip(wide, info.min, info.max)`.

A sum has its own dtype

A sum over an `int8` array does not use `int8`. NumPy sums small integers in the platform integer, which is `int64` here. You can force the narrow type with `dtype=`, and the sum wraps.

```
import numpy as np

a = np.array([100, 120, 127], dtype=np.int8)
print(a.sum(), a.sum().dtype)
print(a.sum(dtype=np.int8))
print(a.mean(), a.mean().dtype)
```

```
347 int64
91
115.66666666666667 float64
```

The true sum is 347. The default sum gives 347. The forced `int8` sum gives 91, which is 347 minus 256. The mean is a `float64` and is correct.

This default is a protection and also a trap. The sum of a `uint8` image is safe. The sum of a `float32` array stays `float32`, and the next section shows that this is not safe.

Floats lose small additions

A `float32` has 24 bits of precision. At 16,777,216, which is 2 to the power 24, the gap between two neighbor values is 2. The sum with 1 is the same number.

```
import numpy as np

print(np.float32(16777216) + np.float32(1))
print(np.float64(16777216) + 1)
print("float32 epsilon:", np.finfo(np.float32).eps)
print("float64 epsilon:", np.finfo(np.float64).eps)
```

```
1.6777216e+07
16777217.0
float32 epsilon: 1.1920929e-07
float64 epsilon: 2.220446049250313e-16
```

The `eps` value is the gap between 1.0 and the next float. For `float32` it is about 1.2e-7, so you get about 7 correct decimal digits. For `float64` you get about 15 or 16.

Now the practical case. Add ten million copies of 0.1 in `float32`. The exact answer is 1,000,000.015, because `float32(0.1)` is a little more than 0.1. Compare three ways to add.

```
import numpy as np

x = np.full(10_000_000, 0.1, dtype=np.float32)
exact = float(np.float32(0.1)) * x.size

pairwise = float(x.sum())
running = float(np.cumsum(x)[-1])
wide = float(x.sum(dtype=np.float64))
for name, value in [("x.sum()", pairwise), ("cumsum[-1]", running), ("sum float64", wide)]:
    print(f"{name:12s} {value:14.3f}  relative error {abs(value - exact) / exact:.2e}")
```

```
x.sum()         1000000.125  relative error 1.10e-07
cumsum[-1]      1087937.000  relative error 8.79e-02
sum float64     1000000.015  relative error 0.00e+00
```

The three results differ a lot. `x.sum()` has a relative error of 1e-7, which is the best a `float32` can give. The cumulative sum from `cumsum` is wrong by almost 9 percent. The cause is that a cumulative sum adds a small number to a large one. Once the total is large, each 0.1 is below the gap between neighbor values and is partly lost.

`x.sum()` is accurate because NumPy sums in pairs. It adds neighbors first, then adds the partial sums, so the numbers it adds stay close in size. `cumsum` cannot do this, because it must produce every cumulative total in order.

A pure Python loop has the same fault as `cumsum`. Chapter 6 uses this fact when it builds a moving average.

Mixed dtypes: which one wins

When two dtypes meet, NumPy picks the smallest dtype that holds both. NumPy 2 added one rule for Python numbers. A Python `int` or `float` does not change the dtype of an array. It takes the dtype of the array, if it fits.

```
import numpy as np

print((np.float32(3) + 3.0).dtype)
print((np.float32(3) + np.float64(3)).dtype)
print((np.int8(1) + np.int16(1)).dtype)
print((np.int64(1) + np.uint64(1)).dtype)
print((np.array([1, 2], dtype=np.uint8) + 1).dtype)
```

```
float32
float64
int16
float64
uint8
```

Read the lines in order. A `float32` plus the Python float 3.0 stays `float32`. A `float32` plus a `float64` becomes `float64`. An `int64` plus a `uint64` has no integer type that holds both, so it becomes `float64`, and large integers lose exactness. The last line is the NumPy 2 rule: a `uint8` array plus the Python integer 1 stays `uint8`.

Older NumPy 1.x chose the dtype from the value of a Python number, so code that you find online can describe different results. Check the version of NumPy before you trust old advice.

What you have now

You have `checked_add`, which turns a silent wrap into an error. You also have four facts that you can check on any machine. A `uint8` add wraps. A `float32` cumulative sum can be wrong by 9 percent. The call `x.sum()` does not have that fault. A mixed expression has a dtype that you can print before you trust it.

The habit that follows is short. Print `.dtype` of every array at the edge of your code, where data comes in, and again where a result goes out.

Chapter 3 moves from what an array holds to who owns it. The next surprise is not a wrong value but a value that changed in a place you did not touch.

Chapter 3 — Views and copies

> **Build** — `root(a)`, a function that finds the array that owns the memory of any view, and a test that shows which operations copy and which do not.
>
> **Theory kept** — a slice is a new description of the same block. An index list or a mask makes a new block. A write through a view changes the original.
>
> **Source** — New. The rules are in the NumPy documentation under "Copies and views". Every output below is from this book's code.
>
> **Run this first** — `uv run python -m examples.ch03_views`, which is `examples/ch03_views.py`.

Chapter 1 showed that `a.T` and `a[:, ::2]` read the block of `a` and copy nothing. This is the reason for the speed of NumPy. It is also the reason for one class of bug: a value that changed in a place that you did not touch.

This chapter gives the rule and builds a function that applies it. It shows two bugs that come from the rule. It also measures a cost that comes from the order of memory.

The rule

Basic slicing makes a view. Indexing with a list of integers or with a boolean mask makes a copy.

```
import numpy as np

a = np.arange(12).reshape(3, 4)
view = a[:, 1:3]       # basic slice
fancy = a[[0, 2]]      # list of integers
masked = a[a > 5]      # boolean mask

print("slice  shares memory:", np.shares_memory(a, view))
print("list   shares memory:", np.shares_memory(a, fancy))
print("mask   shares memory:", np.shares_memory(a, masked))
```

```
slice  shares memory: True
list   shares memory: False
mask   shares memory: False
```

The reason is the strides from chapter 1. A slice is always a regular pattern, so one set of strides can describe it. A list of integers or a mask can pick any elements in any order, and no set of strides can describe that. NumPy must make a new block.

A write through a view changes the original.

```
import numpy as np

a = np.arange(12).reshape(3, 4)
view = a[:, 1:3]
view[0, 0] = -1
print(a[0])
```

```
[ 0 -1  2  3]
```

The value at `a[0, 1]` is now −1, although the code never wrote to `a`.

The function you build

The `.base` attribute of a view points to the array that it looks at. A view of a view points to the array one step up. Follow the chain until it ends.

```
import numpy as np


def root(a: np.ndarray) -> np.ndarray:
    """Follow .base links to the array that owns the memory of `a`."""
    while isinstance(a.base, np.ndarray):
        a = a.base
    return a


a = np.arange(12).reshape(3, 4)
b = a[1:]
c = b[:, 1:]
d = c[[0]]
print("c is a view of:", root(c) is root(a))
print("d is a view of:", root(d) is root(a))
print("root of d is d:", root(d) is d)
```

```
c is a view of: True
d is a view of: False
root of d is d: True
```

The check `isinstance(a.base, np.ndarray)` is needed. An array can be built on top of bytes, a memory-mapped file, or a buffer from another library. In those cases `.base` is not an array, and the loop must stop.

Two arrays share memory when `root(x) is root(y)`. This is a fast test. It is not exact, because two views of one block can still cover different bytes. For the exact answer, use `np.shares_memory`, and see the next block.

```
import numpy as np

a = np.arange(10)
even, odd = a[::2], a[1::2]
print("may_share_memory:", np.may_share_memory(even, odd))
print("shares_memory:   ", np.shares_memory(even, odd))
```

```
may_share_memory: True
shares_memory:    False
```

The function `np.may_share_memory` compares only the first and last byte that each array covers. The two interleaved slices cover the same range, so it says yes. The function `np.shares_memory` solves the exact problem and says no. The exact function can be slow for complex strides. The quick function never says no when the answer is yes.

Bug 1: a function that changes its argument

A function that uses `-=` on its argument writes into the memory of the caller.

```
import numpy as np


def center(x):
    x -= x.mean()
    return x


data = np.array([[1.0, 10.0], [3.0, 20.0], [5.0, 30.0]])
col = data[:, 0]
centered = center(col)
print("data after the call:")
print(data)
```

```
data after the call:
[[-2. 10.]
 [ 0. 20.]
 [ 2. 30.]]
```

The caller passed one column. The column is a view, so the call changed the first column of `data`. The caller sees a different table, and the function did not return it.

The fix is one character: `x = x - x.mean()` makes a new array. The in-place operators `+=`, `-=`, `*=` and `/=` are faster and use less memory. Use them on arrays that your function made itself. Do not use them on an argument unless the name or the documentation says that the function changes it.

Bug 2: a small view that keeps a large block alive

A view keeps its block alive. If you slice 10 rows from a 288 MB array and then delete the large array, the 288 MB stays in memory.

```
import numpy as np

big = np.zeros((6000, 6000))
small = big[:10]
print("small is", small.nbytes, "bytes")
print("small keeps", small.base.nbytes, "bytes alive")

small = small.copy()
print("after copy, base is:", small.base)
```

```
small is 480000 bytes
small keeps 288000000 bytes alive
after copy, base is: None
```

Use `.copy()` when you keep a small piece of a large array. The copy owns 10 rows, and the large array is free to go.

What a copy costs, and what a bad walk costs

Chapter 1 said that a transpose is not in C order. A C-contiguous array has its last axis in consecutive memory. NumPy reads consecutive memory fast, because one read brings in a line of 64 bytes. Strided reads waste most of the line.

Measure two cases on a 6000-by-6000 `float64` array, which is 288 MB. The first is a plain copy against a copy of the transpose. The second is a sum over 600 rows against a sum over 600 columns.

```
import timeit

import numpy as np

n = 6000
big = np.random.default_rng(0).random((n, n))


def best(f, repeat=3):
    return min(timeit.repeat(f, number=1, repeat=repeat))


t_copy = best(lambda: big.copy())
t_tcopy = best(lambda: np.ascontiguousarray(big.T))
t_row = best(lambda: [big[j, :].sum() for j in range(0, n, 10)])
t_col = best(lambda: [big[:, j].sum() for j in range(0, n, 10)])
t_ax0 = best(lambda: big.sum(axis=0))
t_ax1 = best(lambda: big.sum(axis=1))

assert t_tcopy / t_copy > 1.3
assert t_col / t_row > 3
assert 0.33 < t_ax0 / t_ax1 < 3
print(f"copy of transpose / plain copy   {t_tcopy / t_copy:5.1f}x")
print(f"600 column sums / 600 row sums   {t_col / t_row:5.1f}x")
print(f"sum(axis=0) / sum(axis=1)        {t_ax0 / t_ax1:5.1f}x")
```

```
# output varies
copy of transpose / plain copy     2.9x
600 column sums / 600 row sums    22.3x
sum(axis=0) / sum(axis=1)          1.3x
```

The ratios above are from one run on the build machine. The asserts are the claims. The transposed copy is at least 1.3 times slower. The column loop is at least 3 times slower. The two whole-array sums are within a factor of 3 of each other.

The third line is a result that you can find surprising. The sum along axis 0 walks against the memory order, and it is still not slow. NumPy does not add one column at a time. It adds whole rows into a row of partial sums, which is the order that memory has. The built-in reductions know the layout. Your own Python loop over columns does not.

The rule that follows has two parts. Use the built-in reduction when one exists. When you must loop in Python, loop over the first axis, so that each step reads one consecutive row.

Which operations copy

The code below tests each line of the summary table that follows. It raises an error if one line is wrong.

```
import numpy as np

a = np.arange(12).reshape(3, 4)
t = a.T
rows = {
    "a[1:3]":            (a[1:3], True),
    "a[:, ::2]":         (a[:, ::2], True),
    "a.T":               (t, True),
    "a[[0, 2]]":         (a[[0, 2]], False),
    "a[a > 5]":          (a[a > 5], False),
    "a.reshape(4, 3)":   (a.reshape(4, 3), True),
    "a.T.reshape(12)":   (t.reshape(12), False),
    "a.ravel()":         (a.ravel(), True),
    "a.T.ravel()":       (t.ravel(), False),
    "a.flatten()":       (a.flatten(), False),
    "a.copy()":          (a.copy(), False),
    "np.array(a)":       (np.array(a), False),
}
for name, (result, expected) in rows.items():
    shared = np.shares_memory(a, result)
    assert shared == expected, name
    print(f"{name:18s} {'view' if shared else 'copy'}")
```

```
a[1:3]             view
a[:, ::2]          view
a.T                view
a[[0, 2]]          copy
a[a > 5]           copy
a.reshape(4, 3)    view
a.T.reshape(12)    copy
a.ravel()          view
a.T.ravel()        copy
a.flatten()        copy
a.copy()           copy
np.array(a)        copy
```

Operation	Result
`a[1:3]`, `a[:, ::2]`, `a.T`	view
`a[[0, 2]]`, `a[a > 5]`	copy
`a.reshape(...)`	view when strides allow, copy otherwise
`a.ravel()`	view when the array is C-contiguous, copy otherwise
`a.flatten()`, `a.copy()`, `np.array(a)`	always a copy

The call `a.reshape(...)` can change from a view to a copy when the array changes layout. Code that depends on the write-through must not rely on `reshape`. Test with `np.shares_memory` or use `np.may_share_memory`, and do not guess.

What you have now

You have `root(a)`, and you can say which of your arrays share memory. You saw two bugs that come from views. A function changed its argument. A small view kept 288 MB alive.

You also measured the cost of a walk against the memory order. It is 3 times slower for a copy and 22 times slower for a Python loop. The built-in reductions do not have this fault.

Chapter 4 uses the other half of the rule. Indexing with lists and masks makes copies, and writing through them has its own trap.

Chapter 4 — Indexing

> **Build** — `group_sum(keys, values, k)`, which adds up the values for each key, in three versions that you can time against each other.
>
> **Theory kept** — an index with integers or a mask picks elements and makes a copy. A write with an index list applies to each position once. A condition in `np.where` is computed on both sides.
>
> **Source** — New. The behavior is from the NumPy documentation on advanced indexing. Every output below is from this book's code.
>
> **Run this first** — `uv run python -m examples.ch04_indexing`, which is `examples/ch04_indexing.py`.

Chapter 3 said that a list of integers or a mask makes a copy. This chapter shows the four traps that follow from that rule. Each trap produces a wrong result with no error. The chapter then builds a function that counts by key, which is the most common reason for an index with repeated values.

Reading with a mask

A mask is a boolean array of the same shape as the data. A comparison makes one. The mask can count, select and replace.

```
import numpy as np

x = np.array([3, -1, 4, -1, 5, -9, 2, 6])
mask = x < 0
print("mask:    ", mask)
print("count:   ", mask.sum())
print("selected:", x[mask])
print("fraction:", mask.mean())
```

```
mask:     [False  True False  True False  True False False]
count:    3
selected: [-1 -1 -9]
fraction: 0.375
```

The sum of a boolean array counts the `True` values. The mean gives the fraction. Both use the fact that `True` is 1.

Trap 1: a write to a copy is lost

A mask on the left of `=` writes into the array. A mask on the right makes a copy, and a write into the copy is lost.

```
import numpy as np

m = np.arange(10)
m[m > 5][0] = 99
print("chained write:", m)

m[m > 5] = 99
print("direct write: ", m)
```

```
chained write: [0 1 2 3 4 5 6 7 8 9]
direct write:  [ 0  1  2  3  4  5 99 99 99 99]
```

The first line looks like it sets one element to 99. Python runs two steps: `m[m > 5]` makes a copy, and `[0] = 99` writes into that copy. The copy is then deleted. The array `m` did not change, and NumPy did not say so. The second line is one step, and it works.

Trap 2: a repeated index is applied once

Add 1 at each position in an index list. If one position appears three times, you want 3. You get 1.

```
import numpy as np

idx = np.array([1, 1, 3, 1])

a = np.zeros(5, dtype=int)
a[idx] += 1
print("a[idx] += 1  :", a)

b = np.zeros(5, dtype=int)
np.add.at(b, idx, 1)
print("np.add.at    :", b)

print("np.bincount  :", np.bincount(idx, minlength=5))
```

```
a[idx] += 1  : [0 1 0 1 0]
np.add.at    : [0 3 0 1 0]
np.bincount  : [0 3 0 1 0]
```

The statement `a[idx] += 1` is `a[idx] = a[idx] + 1`. NumPy reads `a[idx]` once, adds 1 to each of the four values, and writes four results. Position 1 receives the same result three times. The function `np.add.at` goes one element at a time, so it is correct. The function `np.bincount` counts directly.

Trap 3: `np.where` computes both sides

`np.where(cond, x, y)` picks values from `x` or from `y`. Both `x` and `y` are computed first. A protection that you put in the condition does not stop a bad value in the branch.

```
import warnings

import numpy as np

x = np.array([4.0, 0.0, 2.0])

with warnings.catch_warnings(record=True) as caught:
    warnings.simplefilter("always")
    r = np.where(x != 0, 1 / x, 0.0)
print("result:  ", r)
print("warnings:", [str(w.message) for w in caught])

r2 = np.divide(1, x, out=np.zeros_like(x), where=x != 0)
print("with where=:", r2)
```

```
result:   [0.25 0.   0.5 ]
warnings: ['divide by zero encountered in divide']
with where=: [0.25 0.   0.5 ]
```

The result is right and the warning shows that the division by zero ran. For division the harm is small, because the bad value is not selected. For `np.log` of a negative number, the branch that you did not select gives a warning. For an index that is out of range, it raises an error. The `where=` argument of a ufunc skips the elements where the condition is false. Chapter 6 covers this argument.

Trap 4: the shape of a mixed index

When you mix a slice and an index list in one index, NumPy can move the axes of the result.

```
import numpy as np

A = np.arange(24).reshape(2, 3, 4)
print(A[0, :, [0, 1]].shape)
print(A[:, [0, 1], [0, 1]].shape)
print(A[0][:, [0, 1]].shape)
```

```
(2, 3)
(2, 2)
(3, 2)
```

Read the first line. The index picks `A[0]` and then columns 0 and 1, and you can expect the shape `(3, 2)`. The result is `(2, 3)`. The rule is that the axes that come from index lists go first when a slice separates two advanced indices. In the first line the scalar `0` and the list `[0, 1]` are both advanced indices, and the slice sits between them.

The third line avoids the problem. It indexes in two steps, and the shape is `(3, 2)`. When the shape of a mixed index matters, use two steps or print `.shape` before you rely on it.

The function you build

A group-by sum is the most common use of a repeated index. You have one value per row and one key per row, and you want one total per key. Three versions follow. They must give the same answer.

```
import numpy as np


def group_sum_loop(keys, values, k):
    out = [0.0] * k
    for key, v in zip(keys.tolist(), values.tolist()):
        out[key] += v
    return np.array(out)


def group_sum_at(keys, values, k):
    out = np.zeros(k)
    np.add.at(out, keys, values)
    return out


def group_sum_bincount(keys, values, k):
    return np.bincount(keys, weights=values, minlength=k)


rng = np.random.default_rng(0)
n, k = 1_000_000, 100
keys = rng.integers(0, k, n)
values = rng.random(n)

ref = group_sum_bincount(keys, values, k)
print("loop    matches bincount:", np.allclose(group_sum_loop(keys, values, k), ref))
print("add.at  matches bincount:", np.allclose(group_sum_at(keys, values, k), ref))
print("total of all groups     :", round(float(ref.sum()), 3), "=", round(float(values.sum()),
3))
```

```
loop    matches bincount: True
add.at  matches bincount: True
total of all groups     : 500316.548 = 500316.548
```

The third line is a second check that does not depend on the other functions. The total of the groups must equal the total of all values.

Now time the three versions.

```
import timeit

import numpy as np

rng = np.random.default_rng(0)
n, k = 1_000_000, 100
keys = rng.integers(0, k, n)
values = rng.random(n)


def loop():
    out = [0.0] * k
    for key, v in zip(keys.tolist(), values.tolist()):
        out[key] += v


def at():
    out = np.zeros(k)
    np.add.at(out, keys, values)


def bincount():
    np.bincount(keys, weights=values, minlength=k)


t = {f.__name__: min(timeit.repeat(f, number=1, repeat=3)) for f in (loop, at, bincount)}
assert t["loop"] / t["at"] > 5
assert t["loop"] / t["bincount"] > 5
for name, sec in t.items():
    print(f"{name:9s} {sec * 1e3:8.2f} ms")
```

```
# output varies
loop        113.62 ms
at            2.74 ms
bincount      2.23 ms
```

The loop is 40 to 50 times slower than either NumPy version. That is the expected result. The surprise is the second line against the third line.

Old advice says that `np.add.at` is very slow and that you must use `np.bincount`. That advice was true for NumPy 1.x. On NumPy 2.5.4 the two are within 25 percent of each other on this input.

Check the version of NumPy before you accept advice about speed. The `bincount` version has one restriction. The keys must be non-negative integers. The function `np.add.at` accepts any index that is valid for the output array.

Finding the largest values without a sort

A full sort puts every value in order. To find the 10 largest of 5 million values, you do not need that. `np.partition` puts the 10 largest at the end, in any order, and it does less work.

```
import timeit

import numpy as np

x = np.random.default_rng(1).random(5_000_000)
top_sort = np.sort(x)[-10:]
top_part = np.sort(np.partition(x, -10)[-10:])
print("same ten values:", np.array_equal(top_sort, top_part))

t_sort = min(timeit.repeat(lambda: np.sort(x)[-10:], number=1, repeat=3))
t_part = min(timeit.repeat(lambda: np.partition(x, -10)[-10:], number=1, repeat=3))
assert t_sort / t_part > 2
print(f"sort / partition: {t_sort / t_part:.1f}x")
```

```
# output varies
same ten values: True
sort / partition: 4.7x
```

The partition result is not in order, so the code sorts the 10 values at the end. The saving is a factor of about 5 on the build machine. The assert requires at least 2.

What you have now

You have three versions of `group_sum` that give the same answer, and you know which one to use first: `np.bincount` when the keys are small non-negative integers, and `np.add.at` for the other cases. You know four traps by name.

- A chained write is lost.
- A repeated index is applied once.
- The `np.where` function computes both sides.
- A mixed index moves axes.

Chapter 5 uses indexing in the other direction. It does not pick elements from an array. It stretches arrays so that they combine without any loop.

Chapter 5 — Broadcasting

> **Build** — three functions that compute all distances between two sets of points, a loop version, a broadcast version and a dot-product version, with a measured memory cost and a measured precision limit for each.
>
> **Theory kept** — two shapes combine when, from the right, each pair of lengths is equal or one of them is 1. A length of 1 is stretched with a stride of 0, so the stretch costs no memory until an operation writes a result.
>
> **Source** — New. The rule is in the NumPy documentation under "Broadcasting". Every output below is from this book's code.
>
> **Run this first** — `uv run python -m examples.ch05_broadcasting`, which is `examples/ch05_broadcasting.py`.

Broadcasting is the rule that lets `a - b` work when `a` and `b` have different shapes. It replaces most of the loops that you write in plain Python. It also makes one new mistake easy: a result that is much larger than you meant.

This chapter states the rule, shows the stride trick behind it, and uses it to compute all pairwise distances. The distance example is the right size to show the three costs of a vectorized program: time, memory and precision.

The rule

Write the two shapes one above the other, aligned on the right. For each column, the two lengths must be equal, or one of them must be 1. A missing length on the left counts as 1. The result has the larger length in each column.

```
import numpy as np

print(np.broadcast_shapes((3, 1), (1, 4)))
print(np.broadcast_shapes((8, 1, 6, 1), (7, 1, 5)))
print(np.broadcast_shapes((5, 3), (3,)))

try:
    np.ones(3) + np.ones(4)
except ValueError as e:
    print("error:", e)
```

```
(3, 4)
(8, 7, 6, 5)
(5, 3)
error: operands could not be broadcast together with shapes (3,) (4,)
```

Read the second line with the rule. The shapes `(8, 1, 6, 1)` and `(7, 1, 5)` align as columns `8 1 6 1` over `_ 7 1 5`. The columns pair as 1 with 5, 6 with 1, 1 with 7, and 8 with nothing. The result is `(8, 7, 6, 5)`. The last line shows what happens when two lengths differ and neither is 1.

Use `None` or `np.newaxis` to add a length of 1 at any position. This is the tool that you use to line up the axes that you want.

```
import numpy as np

x = np.arange(3)
print((x[:, None] - x[None, :]))
print((x[:, None] - x[None, :]).shape)
```

```
[[ 0 -1 -2]
 [ 1  0 -1]
 [ 2  1  0]]
(3, 3)
```

The column vector `x[:, None]` has shape `(3, 1)`, and the row vector `x[None, :]` has shape `(1, 3)`. The result `(3, 3)` holds every difference `x[i] - x[j]`.

A stretch costs no memory

NumPy does not copy a length-1 axis to make it longer. It sets the stride of that axis to 0, so every step reads the same element. This is the idea from chapter 1, used again.

```
import numpy as np

row = np.arange(3.0)
stretched = np.broadcast_to(row, (1000, 3))
print("shape  ", stretched.shape)
print("strides", stretched.strides)
print("block  ", stretched.base.nbytes, "bytes")
print("result of an add:", (stretched + 1).nbytes, "bytes")
```

```
shape   (1000, 3)
strides (0, 8)
block   24 bytes
result of an add: 24000 bytes
```

The stretched array looks like 1000 rows. It has a block of 24 bytes and a stride of 0 along the first axis. The cost appears when an operation makes a result, because the result must have all 1000 rows.

The trap: a result you did not mean

A vector of shape `(n,)` and a column of shape `(n, 1)` broadcast to `(n, n)`. NumPy gives no warning, because the rule is satisfied.

```
import numpy as np

rng = np.random.default_rng(0)
values = rng.random(1000)
column = rng.random((1000, 1))

r = values - column
print("shape:", r.shape)
print("size: ", r.nbytes / 1e6, "MB")

X = rng.random((5, 3))
try:
    X - X.mean(axis=1)
except ValueError as e:
    print("error:", e)
print("with keepdims:", (X - X.mean(axis=1, keepdims=True)).shape)
```

```
shape: (1000, 1000)
size:  8.0 MB
error: operands could not be broadcast together with shapes (5,3) (5,)
with keepdims: (5, 3)
```

For `n = 1000` the mistake costs 8 MB, and nobody notices. For `n = 100,000` the same mistake needs 80 GB. The usual cause is a function that returned `(n, 1)` where you expected `(n,)`. An example is a `reshape(-1, 1)` that you added for another library.

The third part of the block is the correct way to subtract a row mean. The call `X.mean(axis=1)` has shape `(5,)`, which does not align with `(5, 3)`. The argument `keepdims=True` keeps the length-1 axis, so the shape is `(5, 1)` and the subtraction works.

The habit that follows: assert the shape of a result at a boundary. A line such as `assert r.shape == (n,)` is cheap and catches this mistake at once.

The function you build

Given `A` with `n` points and `B` with `m` points, in `d` dimensions, build the `(n, m)` matrix of distances. Three versions follow.

```
import numpy as np


def dist_loop(A, B):
    out = np.empty((len(A), len(B)))
    for i, a in enumerate(A):
        for j, b in enumerate(B):
            out[i, j] = np.sqrt(((a - b) ** 2).sum())
    return out


def dist_broadcast(A, B):
    """Exact. Builds an (n, m, d) temporary array."""
    return np.sqrt(((A[:, None, :] - B[None, :, :]) ** 2).sum(-1))


def dist_dot(A, B):
    """Fast and small, but loses precision when points are close together and far from the
origin."""
    d2 = (A ** 2).sum(1)[:, None] + (B ** 2).sum(1)[None, :] - 2 * A @ B.T
    return np.sqrt(np.maximum(d2, 0))


rng = np.random.default_rng(0)
A, B = rng.random((300, 16)), rng.random((400, 16))
ref = dist_loop(A, B)
print("shape:", ref.shape)
print("broadcast max error:", np.abs(dist_broadcast(A, B) - ref).max())
print("dot       max error:", np.abs(dist_dot(A, B) - ref).max())
```

```
shape: (300, 400)
broadcast max error: 0.0
dot       max error: 2.886579864025407e-15
```

The broadcast version is the same formula as the loop, with `A[:, None, :]` of shape `(n, 1, d)` and `B[None, :, :]` of shape `(1, m, d)`. It subtracts to `(n, m, d)`, squares, and sums over the last axis. The dot version uses the identity that the squared distance is `|a|² + |b|² − 2 a·b`. The cross term `A @ B.T` is one matrix product.

On this well-behaved input, all three agree. The dot version differs from the loop by about 3e-15, which is the rounding of `float64`.

Cost 1: time

```
import timeit

import numpy as np


def dist_loop(A, B):
    out = np.empty((len(A), len(B)))
    for i, a in enumerate(A):
        for j, b in enumerate(B):
            out[i, j] = np.sqrt(((a - b) ** 2).sum())
    return out


def dist_broadcast(A, B):
    return np.sqrt(((A[:, None, :] - B[None, :, :]) ** 2).sum(-1))


def dist_dot(A, B):
    d2 = (A ** 2).sum(1)[:, None] + (B ** 2).sum(1)[None, :] - 2 * A @ B.T
    return np.sqrt(np.maximum(d2, 0))


rng = np.random.default_rng(0)
A, B = rng.random((300, 16)), rng.random((400, 16))
t = {f.__name__: min(timeit.repeat(lambda: f(A, B), number=1, repeat=3))
     for f in (dist_loop, dist_broadcast, dist_dot)}
assert t["dist_loop"] / t["dist_broadcast"] > 10
assert t["dist_broadcast"] / t["dist_dot"] > 2
for name, sec in t.items():
    print(f"{name:15s} {sec * 1e3:9.2f} ms")
```

```
# output varies
dist_loop         279.18 ms
dist_broadcast      8.76 ms
dist_dot            0.75 ms
```

The broadcast version is about 30 times faster than the loop. The dot version is about 10 times faster again. The asserts require at least 10 and at least 2. The dot version wins because a matrix product uses an optimized library routine and because it does less work: it never forms the `(n, m, d)` array.

Cost 2: memory

The broadcast version builds an array with `n * m * d` values. For 2000 points in 64 dimensions, that is 256 million values, which is 2 GB.

```
import tracemalloc

import numpy as np


def peak_mb(f, *args):
    tracemalloc.start()
    f(*args)
    _, peak = tracemalloc.get_traced_memory()
    tracemalloc.stop()
    return peak / 1e6


def dist_broadcast(A, B):
    return np.sqrt(((A[:, None, :] - B[None, :, :]) ** 2).sum(-1))


def dist_dot(A, B):
    d2 = (A ** 2).sum(1)[:, None] + (B ** 2).sum(1)[None, :] - 2 * A @ B.T
    return np.sqrt(np.maximum(d2, 0))


rng = np.random.default_rng(0)
A, B = rng.random((2000, 64)), rng.random((2000, 64))
print("result alone:       ", 2000 * 2000 * 8 / 1e6, "MB")
print("broadcast peak:     ", round(peak_mb(dist_broadcast, A, B)), "MB")
print("dot peak:           ", round(peak_mb(dist_dot, A, B)), "MB")
```

```
result alone:        32.0 MB
broadcast peak:      2080 MB
dot peak:            96 MB
```

The result itself is 32 MB. The dot version peaks at 96 MB, which is 3 times the size of the result. The broadcast version peaks at 2080 MB, which is 65 times the size of the result. The temporary array grows with `n * m * d`. At 3000 points it alone has 3000 × 3000 × 64 × 8 bytes, which is 4.6 GB.

Two remedies exist for the broadcast version. You can process `A` in blocks of rows, so that the temporary array has a fixed size. Or you can use the dot version, and read the next section first.

Cost 3: precision

The dot version subtracts two large numbers that are almost equal. The squared norms `|a|²` and `|b|²` are large when the points are far from the origin. The squared distance is small when the points are close to each other. The difference loses digits.

```
import numpy as np


def dist_loop(A, B):
    out = np.empty((len(A), len(B)))
    for i, a in enumerate(A):
        for j, b in enumerate(B):
            out[i, j] = np.sqrt(((a - b) ** 2).sum())
    return out


def dist_broadcast(A, B):
    return np.sqrt(((A[:, None, :] - B[None, :, :]) ** 2).sum(-1))


def dist_dot(A, B):
    d2 = (A ** 2).sum(1)[:, None] + (B ** 2).sum(1)[None, :] - 2 * A @ B.T
    return np.sqrt(np.maximum(d2, 0))


def dist_dot_centered(A, B):
    c = np.concatenate([A, B]).mean(axis=0)
    return dist_dot(A - c, B - c)


rng = np.random.default_rng(0)
P = rng.random((60, 3)) * 0.01 + 1e4        # 60 points inside a cube of side 0.01, at 10,000
ref = dist_loop(P, P)
print("mean true distance:      ", round(float(ref.mean()), 4))
print("dot            max error:", f"{np.abs(dist_dot(P, P) - ref).max():.1e}")
print("dot centered   max error:", f"{np.abs(dist_dot_centered(P, P) - ref).max():.1e}")

P32 = P.astype(np.float32)
print("float32, largest distance, true:    ", round(float(dist_broadcast(P32, P32).max()), 4))
print("float32, largest distance, dot:     ", float(dist_dot(P32, P32).max()))
print("float32, largest distance, centered:", round(float(dist_dot_centered(P32, P32).max()),
4))
```

```
mean true distance:       0.0069
dot            max error: 3.5e-04
dot centered   max error: 1.2e-10
float32, largest distance, true:     0.0144
float32, largest distance, dot:      8.0
float32, largest distance, centered: 0.0144
```

The first three lines are for `float64`. The error of the plain dot version is 3.5e-4, and the mean true distance is 6.9e-3. The error is 5 percent of the answer. After centering, the error is 1.2e-10.

The `float32` lines are worse. The squared norms are about 3e8. At that size the gap between neighbor `float32` values is 32. The plain dot version reports a largest distance of 8.0. The true value is 0.0144. The answer is wrong by a factor of more than 500.

The centered version is correct. After the origin moves to the middle of the data, the numbers are small. Then `float32` has digits to spare.

Centering is the fix to remember. It costs one mean and two subtractions. It does not help when the points are spread over a large range. In that case the loss is a property of the problem and not of the method.

Which one to use

Version	Time	Peak memory	Precision
`dist_loop`	slowest by far	the result	exact
`dist_broadcast`	middle	`n * m * d` values	exact
`dist_dot_centered`	fastest	a few results	good when centered

Use the dot version with centering as the default, and test it against the broadcast version on a small sample of your data. Use the broadcast version when you need the exact differences, for example when you also need the direction `a - b`. Use the loop only to check the others.

What you have now

You have the broadcasting rule and `np.broadcast_shapes` to test it. You know that a stretch is a stride of 0 and costs nothing until a result is made. You have three distance functions, and for each one you have a measured time, a measured peak memory and a measured error.

The lesson of the chapter is that the fastest version of an array program is also the one that loses the most precision. Measure all three costs before you choose.

Chapter 6 turns to the other half of a vectorized program: the ufunc, its `reduce` and `accumulate` methods, and the cases where no vectorized form exists.

Chapter 6 — Ufuncs and vectorization

> **Build** — a moving average in four versions, a loop, a cumulative-sum version, a window version and a convolution, with a measured time and a measured precision for each.
>
> **Theory kept** — a ufunc is a function that runs element by element in compiled code. It has methods (`reduce` , `accumulate` , `outer`) that apply it along an axis. Vectorization is the act of rewriting a loop as calls to ufuncs. It does not exist for every loop.
>
> **Source** — New. The ufunc methods are in the NumPy documentation. Every output below is from this book's code.
>
> **Run this first** — `uv run python -m examples.ch06_ufuncs` , which is `examples/ch06_ufuncs.py` .

A ufunc, short for universal function, is the unit of speed in NumPy. `np.add` , `np.multiply` , `np.sqrt` , `np.maximum` and about 90 others are ufuncs. The operators `+` and `*` call them. A ufunc takes arrays, loops over the elements in compiled code, and returns an array. It does the Python-level work once for the whole array, and not once per element.

This chapter shows three methods that make a ufunc more than an element-wise function. It shows one wrong way to get the speed. It ends with a moving average that you build four times.

Three methods on every binary ufunc

A binary ufunc has a `reduce` method, an `accumulate` method and an `outer` method. Each one applies the ufunc in a different pattern.

```
import numpy as np

x = np.array([3, 1, 4, 1, 5, 9, 2, 6])
print("reduce     :", np.add.reduce(x))
print("accumulate :", np.add.accumulate(x))
print("running max:", np.maximum.accumulate(x))
print("outer      :")
print(np.multiply.outer(x[:3], x[:3]))
```

```
reduce     : 31
accumulate : [ 3  4  8  9 14 23 25 31]
running max: [3 3 4 4 5 9 9 9]
outer      :
[[ 9  3 12]
 [ 3  1  4]
 [12  4 16]]
```

The call `np.add.reduce(x)` is `x.sum()` . It folds the array to one value. The call `np.add.accumulate(x)` is `np.cumsum(x)` . It keeps every partial result. The call `np.maximum.accumulate(x)` gives the running maximum, which has no special name in NumPy. The call `np.multiply.outer` gives every product of a pair, which is the broadcast of chapter 5 written as a method.

The methods let you build many functions from the 90 ufuncs. A running minimum, a running product and a running logical `or` are one call each.

axis, keepdims, out and where

Every reduction takes `axis`, which can be an integer or a tuple. It takes `keepdims`, which chapter 5 used to keep the reduced axis with length 1. Every ufunc takes `out`, to write the result into an array that you made, and `where`, to skip some elements.

```
import numpy as np

m = np.arange(6).reshape(2, 3)
print("axis 0      :", m.sum(axis=0))
print("axis 1      :", m.sum(axis=1))
print("axes (0, 1) :", m.sum(axis=(0, 1)))
print("keepdims    :", m.sum(axis=1, keepdims=True).shape)

y = np.array([1.0, -1.0, 4.0])
out = np.full(3, np.nan)
np.sqrt(y, out=out, where=y >= 0)
print("sqrt with where:", out)
```

```
axis 0      : [3 5 7]
axis 1      : [ 3 12]
axes (0, 1) : 15
keepdims    : (2, 1)
sqrt with where: [ 1. nan  2.]
```

The last line shows the contract of `where`. The element that the condition skipped keeps its old value in `out`. If you do not pass `out`, the values of the skipped elements are undefined, so always pass it.

One more behavior has a large effect on real data. A `nan` spreads through a sum.

```
import numpy as np

x = np.array([1.0, np.nan, 2.0])
print("sum   :", x.sum())
print("nansum:", np.nansum(x))
print("mean  :", x.mean(), "nanmean:", np.nanmean(x))
```

```
sum   : nan
nansum: 3.0
mean  : nan nanmean: 1.5
```

The `nan` functions skip the missing values. Use them with intent. A silent `nansum` on a column that is mostly `nan` gives a number, and the number is a sum of very few values.

Temporaries and `out=`

An expression such as `a * b + c` makes a temporary array for `a * b`, and then a second array for the sum. For small arrays this does not matter. For large arrays, each temporary is a pass over memory and an allocation. Writing into a result array that you made in advance avoids both.

```
import timeit

import numpy as np

rng = np.random.default_rng(1)
a = rng.random(10_000_000)
b = rng.random(10_000_000)
c = rng.random(10_000_000)
out = np.empty_like(a)


def expression():
    return a * b + c


def in_place():
    np.multiply(a, b, out=out)
    np.add(out, c, out=out)
    return out


assert np.array_equal(expression(), in_place())
t_expr = min(timeit.repeat(expression, number=1, repeat=5))
t_inpl = min(timeit.repeat(in_place, number=1, repeat=5))
assert t_expr / t_inpl > 1.1
print(f"expression / in place: {t_expr / t_inpl:.2f}x")
```

```
# output varies
expression / in place: 1.41x
```

The gain on the build machine is 1.4 times, and the assert requires 1.1. This is a modest gain, and the code is harder to read. Use `out=` when the profile shows that this line matters, and not before.

`np.vectorize` is not vectorization

The name suggests speed. The documentation says otherwise: the function is provided for convenience, and it is a Python loop. Measure it against a plain loop and against the real vectorized form.

```
import timeit

import numpy as np

v = np.random.default_rng(0).random(1_000_000)


def f(a):
    return a * a + 1 if a > 0.5 else a


vf = np.vectorize(f)
t_loop = min(timeit.repeat(lambda: [f(a) for a in v.tolist()], number=1, repeat=3))
t_vec = min(timeit.repeat(lambda: vf(v), number=1, repeat=3))
t_where = min(timeit.repeat(lambda: np.where(v > 0.5, v * v + 1, v), number=1, repeat=3))

assert t_vec / t_loop > 0.5          # np.vectorize is no faster than the loop
assert t_loop / t_where > 5          # the real vectorized form is
print(f"list comprehension {t_loop * 1e3:7.1f} ms")
print(f"np.vectorize       {t_vec * 1e3:7.1f} ms")
print(f"np.where           {t_where * 1e3:7.1f} ms")
```

```
# output varies
list comprehension   166.1 ms
np.vectorize         203.4 ms
np.where             15.1 ms
```

The `np.vectorize` call is slower than the list comprehension that it replaces, on the build machine. The real vectorized form, with `np.where`, is 11 to 13 times faster than both. Use `np.vectorize` to make a Python function accept arrays in a place where speed does not matter. Do not use it to go faster.

The function you build: a moving average

A moving average of window `w` replaces each value with the mean of the `w` values that end there. The loop version is the definition. Three others follow.

```
import numpy as np
from numpy.lib.stride_tricks import sliding_window_view


def ma_loop(x, w):
    out = np.empty(len(x) - w + 1)
    for i in range(len(out)):
        out[i] = x[i:i + w].mean()
    return out


def ma_cumsum(x, w):
    c = np.concatenate([[0.0], np.cumsum(x)])
    return (c[w:] - c[:-w]) / w


def ma_window(x, w):
    return sliding_window_view(x, w).mean(axis=-1)


def ma_convolve(x, w):
    return np.convolve(x, np.ones(w) / w, mode="valid")


x = np.random.default_rng(2).random(20_000)
ref = ma_loop(x, 50)
for f in (ma_cumsum, ma_window, ma_convolve):
    print(f"{f.__name__:11s} max difference from the loop: {np.abs(f(x, 50) -
ref).max():.1e}")
```

```
ma_cumsum   max difference from the loop: 2.4e-13
ma_window   max difference from the loop: 0.0e+00
ma_convolve max difference from the loop: 3.3e-16
```

All three agree with the loop to within rounding. The version `ma_cumsum` is based on one fact: the sum of the window that ends at `i` is the cumulative sum at `i` minus the cumulative sum `w` steps earlier. This is two passes over the data, whatever the size of `w`. The version `ma_window` uses `sliding_window_view`, which makes an `(n - w + 1, w)` view without copying, as chapter 3 explained, and then takes the mean of each row. The version `ma_convolve` treats the average as a convolution with a flat kernel.

Time them on one million values with `w = 50`.

```
import timeit

import numpy as np
from numpy.lib.stride_tricks import sliding_window_view


def ma_loop(x, w):
    out = np.empty(len(x) - w + 1)
    for i in range(len(out)):
        out[i] = x[i:i + w].mean()
    return out


def ma_cumsum(x, w):
    c = np.concatenate([[0.0], np.cumsum(x)])
    return (c[w:] - c[:-w]) / w


def ma_window(x, w):
    return sliding_window_view(x, w).mean(axis=-1)


def ma_convolve(x, w):
    return np.convolve(x, np.ones(w) / w, mode="valid")


x = np.random.default_rng(2).random(1_000_000)
t_loop = min(timeit.repeat(lambda: ma_loop(x[:20_000], 50), number=1, repeat=3)) * 50   #
scaled to 1,000,000
t = {f.__name__: min(timeit.repeat(lambda: f(x, 50), number=1, repeat=3))
     for f in (ma_cumsum, ma_window, ma_convolve)}
assert t_loop / t["ma_cumsum"] > 20
print(f"ma_loop (scaled from 20,000 values) {t_loop * 1e3:8.0f} ms")
for name, sec in t.items():
    print(f"{name:35s} {sec * 1e3:8.0f} ms")
```

```
# output varies
ma_loop (scaled from 20,000 values)     2747 ms
ma_cumsum                                 16 ms
ma_window                                 44 ms
ma_convolve                               17 ms
```

The loop is measured on 20,000 values and multiplied by 50, so that the run is short. The vectorized versions are 60 to 170 times faster. The window version is the slowest of the three because it computes a real mean over a `w`-wide row for every output. The cumulative-sum version and the convolution are about the same speed.

The precision trap in the fast version

The cumulative-sum version is the fastest and the shortest. It is also the one that chapter 2 warned about. Its cumulative sum grows large, and every window sum is the difference of two large numbers.

```
import numpy as np
from numpy.lib.stride_tricks import sliding_window_view


def ma_cumsum(x, w):
    c = np.concatenate([[0.0], np.cumsum(x)])
    return (c[w:] - c[:-w]) / w


def ma_cumsum_centered(x, w):
    m = x.mean()
    return ma_cumsum(x - m, w) + m


def ma_window(x, w):
    return sliding_window_view(x, w).mean(axis=-1)


print("float32 data, values from 10000 to 10100, one million values")
for seed in (2, 3):
    s = np.random.default_rng(seed).random(1_000_000)
    x32 = (s * 100 + 1e4).astype(np.float32)
    truth = ma_window(x32.astype(np.float64), 50)
    for f in (ma_cumsum, ma_cumsum_centered, ma_window):
        print(f"seed {seed} {f.__name__:20s} max error {np.abs(f(x32, 50) -
truth).max():.3g}")
    x64 = x32.astype(np.float64)
    print(f"seed {seed} {'ma_cumsum in float64':20s} max error {np.abs(ma_cumsum(x64, 50) -
truth).max():.3g}")
```

```
float32 data, values from 10000 to 10100, one million values
seed 2 ma_cumsum            max error 207
seed 2 ma_cumsum_centered   max error 0
seed 2 ma_window            max error 0.00197
seed 2 ma_cumsum in float64 max error 0
seed 3 ma_cumsum            max error 209
seed 3 ma_cumsum_centered   max error 0.000469
seed 3 ma_window            max error 0.00186
seed 3 ma_cumsum in float64 max error 0
```

The plain cumulative-sum version has an error of about 200 on values near 10,000. That is 2 percent of the values. The window version has an error of 0.002, which is about twice the spacing of `float32` at that size.

The centered version subtracts the mean first, so the cumulative sum stays small. On seed 2 its error is exactly 0. This is a property of that data and not of the method. The centered total never leaves the range where `float32` can hold it exactly, because every value is a multiple of 2^{-10}.

On seed 3 the error is 4.7e-4, which is smaller than the window version.

Read the two seeds together. Centering gives a good answer in both cases, and the exact zero on one seed is luck.

If you do not know the range of your data, use the window version or use `float64` for the cumulative sum. The last line of each seed shows the `float64` result. Its error is 0 on both seeds, for the same reason as the exact zero above. The values sit on a grid of 2^{-10}, and `float64` holds their sums exactly.

When no vectorized form exists

Some loops cannot be written with ufuncs. The usual cause is a dependency: each value needs the result for the previous value. An exponential moving average is the standard case.

```
import time

import numpy as np


def ema_loop(x, alpha):
    out = np.empty_like(x)
    acc = x[0]
    for i, v in enumerate(x.tolist()):
        acc = alpha * v + (1 - alpha) * acc
        out[i] = acc
    return out


x = np.random.default_rng(2).random(1_000_000)
start = time.perf_counter()
y = ema_loop(x, 0.1)
elapsed = time.perf_counter() - start
assert np.isfinite(y[-1]) and elapsed < 10
print(f"{len(x) / elapsed / 1e6:.1f} million steps a second")
```

```
# output varies
3.6 million steps a second
```

The loop runs at 3.6 million steps a second on the build machine. There is no `np.add.accumulate` form of this, because the weight of each step depends on the running value. Two honest options exist outside this book. A filter routine from a signal library applies this recurrence in compiled code. A compiler such as Numba turns the loop into machine code. This book did not measure either, so it makes no claim about their speed.

What you have now

You have the three ufunc methods and four arguments: `axis`, `keepdims`, `out` and `where`. You have a moving average in four versions, with a time and an error for each. You know that `np.vectorize` is a loop. You know that `out=` saves a factor of 1.4 on large arrays. You know that the cumulative-sum trick needs centered data in `float32`.

The test that you can apply to any loop is short. If each iteration needs only its own element and its neighbors at fixed offsets, a vectorized form exists. If an iteration needs the result of the previous one, it probably does not.

Chapter 7 uses NumPy for the two jobs that most users meet next: solving linear systems and drawing random numbers that you can reproduce.

Chapter 7 — Linear algebra and random numbers

> **Build** — `fit_line(x, y)`, a least-squares line with a bootstrap interval for the slope, which gives the same answer every time that you give it the same seed.
>
> **Theory kept** — solve a system with `solve` and not with the inverse. Fit with `lstsq` and not with the normal equations. Make random numbers from a `Generator` that you create and pass, and not from a global state.
>
> **Source** — New. The conditioning rule is standard numerical linear algebra. The random-number policy is from the NumPy documentation. Every output below is from this book's code.
>
> **Run this first** — `uv run python -m examples.ch07_linalg_random`, which is `examples/ch07_linalg_random.py`.

Two jobs account for most of the NumPy code outside data cleaning. One is to solve for unknown numbers, such as the coefficients of a fit. The other is to draw random numbers, for a simulation, a split or a resample. Both have a textbook form that works on small examples and fails on real ones. This chapter measures how each one fails.

The matrix product

The operator `@` is the matrix product. It works on vectors, matrices and stacks of matrices.

```
import numpy as np

A = np.arange(6.0).reshape(2, 3)
v = np.array([1.0, 0.0, -1.0])
stack = np.arange(24.0).reshape(4, 2, 3)

print("matrix @ vector      :", (A @ v).shape)
print("stack of 4 @ vector  :", (stack @ v).shape)
print("stack of 4 @ A.T     :", (stack @ A.T).shape)
```

```
matrix @ vector      : (2,)
stack of 4 @ vector  : (4, 2)
stack of 4 @ A.T     : (4, 2, 2)
```

The last two lines show the rule. Leading axes act as a stack, and `@` multiplies the last two axes of each matrix. The leading axes broadcast, as in chapter 5. One call does four matrix products.

Use `@` and `np.dot` only on arrays. Do not use the `*` operator for a matrix product. It multiplies element by element, and for two square matrices it gives a result with the right shape and the wrong values.

Solve, do not invert

To solve `A x = b` you can compute `inv(A) @ b` or call `solve(A, b)`. Both give the same answer in a textbook. Test them on a Hilbert matrix, which is a standard badly conditioned matrix. The right answer is a vector of ones, because

the code builds `b` from it.

```
import numpy as np


def hilbert(n):
    i = np.arange(n)
    return 1.0 / (i[:, None] + i[None, :] + 1)


print(f"{'n':>3} {'cond(A)':>9} {'solve error':>12} {'inv error':>10} {'solve resid':>12}
{'inv resid':>10}")
for n in (4, 8, 12):
    H = hilbert(n)
    true = np.ones(n)
    b = H @ true
    x_solve = np.linalg.solve(H, b)
    x_inv = np.linalg.inv(H) @ b
    print(f"{n:>3} {np.linalg.cond(H):9.1e} {np.abs(x_solve - true).max():12.1e} "
          f"{np.abs(x_inv - true).max():10.1e} {np.abs(H @ x_solve - b).max():12.1e} "
          f"{np.abs(H @ x_inv - b).max():10.1e}")
```

```
  n   cond(A)  solve error  inv error  solve resid  inv resid
  4   1.6e+04      6.6e-14    9.1e-13      0.0e+00    9.5e-14
  8   1.5e+10      1.2e-07    1.9e-06      2.2e-16    1.4e-08
 12   1.8e+16      7.0e-01    2.1e+01      4.4e-16    5.4e-02
```

Read the table in three parts.

The condition number `cond(A)` measures how much the answer can change when the input changes a little. A rule of thumb is that the error of the answer is about `cond(A)` times the rounding unit of `float64`, which is 1.1e-16. For `n = 8`, that is 1.5e10 times 1.1e-16, or about 2e-6. The table shows 1.2e-7 for `solve`, and 1.9e-6 for `inv`. The rule gives the order of magnitude.

For `n = 12` the condition number is 1.8e16, which is more than the number of digits that `float64` has. No method can give a good answer. Both are wrong, and the errors are 0.7 and 21.

The last two columns show the real difference. The residual `H @ x - b` of `solve` is at the rounding level for every `n`, even when `x` is wrong. The residual of `inv` is 6e7 times larger at `n = 8` and 1e14 times larger at `n = 12`. The inverse method gives an answer that does not even satisfy the equation. Use the residual as a check: it tells you if the method did its job, where the error tells you only that the problem was hard.

Speed points the same way. For a 2000-by-2000 system, `solve` took 0.18 s and `inv(A) @ b` took 0.48 s on the build machine. The inverse does more work for a worse answer.

Fit with `lstsq`, not with the normal equations

A least-squares fit minimizes `|X β − y|²`. The textbook solution is the normal equations, `(XᵀX) β = Xᵀy`. The problem is that `XᵀX` has the square of the condition number of `X`. Compare the two on polynomial fits, where `y` is built from known coefficients of 1.

```
import numpy as np

x = np.linspace(0, 1, 50)
print(f"{'degree':>6} {'cond(X)':>9} {'lstsq error':>12} {'normal-eq error':>16}")
for degree in (3, 6, 9, 12):
    X = np.vander(x, degree + 1)
    y = X @ np.ones(degree + 1)
    via_lstsq = np.linalg.lstsq(X, y, rcond=None)[0]
    via_normal = np.linalg.solve(X.T @ X, X.T @ y)
    print(f"{degree:>6} {np.linalg.cond(X):9.1e} {np.abs(via_lstsq - 1).max():12.1e} "
          f"{np.abs(via_normal - 1).max():16.1e}")
```

```
degree   cond(X)  lstsq error  normal-eq error
     3   1.2e+02      7.1e-15          1.4e-12
     6   2.0e+04      8.1e-13          4.1e-08
     9   3.6e+06      5.4e-11          1.3e-03
    12   6.8e+08      9.7e-09          3.9e-01
```

At degree 9 the error of the normal equations is 1.3e-3 and the error of `lstsq` is 5.4e-11. The normal equations lose 7 digits more. At degree 12 the error is 0.39 where `lstsq` has 1e-8. The cause is the squared condition number: `cond(X)` of 6.8e8 at degree 12 becomes 4.6e17 for X^TX, which is past the limit of `float64`. The function `lstsq` uses a factorization of `X` that does not form X^TX.

The function also returns the rank and the singular values, and that is its second use. If the rank is less than the number of columns, two of your columns carry the same information, and the coefficients are not unique. Check the rank of any design matrix that you did not build yourself.

Random numbers: create a generator and pass it

NumPy has two random-number interfaces. The old one is the global `np.random.seed` and `np.random.rand`. The current one is `np.random.default_rng`, which returns a `Generator` object. Use the current one.

```
import numpy as np

a = np.random.default_rng(42).random(3)
b = np.random.default_rng(42).random(3)
c = np.random.default_rng(43).random(3)
print("seed 42:", a)
print("seed 42:", b)
print("seed 43:", c)
print("same seed, same numbers:", np.array_equal(a, b))
```

```
seed 42: [0.77395605 0.43887844 0.85859792]
seed 42: [0.77395605 0.43887844 0.85859792]
seed 43: [0.65229926 0.04377532 0.02002959]
same seed, same numbers: True
```

The reason to avoid the global state is not style. Every function in the process uses the same global state. If one library function draws a number between your two draws, your sequence changes, and the cause is far from the symptom. A

`Generator` that you create and pass to the functions that need it has no such coupling. The function signature shows that the function is random, and the caller controls the seed.

Parallel streams: `spawn`, not `seed + i`

A job with several workers needs one stream per worker, and the streams must not overlap or repeat. The tool is `SeedSequence.spawn`.

```
import numpy as np


def work(seed_seq):
    return np.random.default_rng(seed_seq).normal(size=1000).mean()


children = np.random.SeedSequence(7).spawn(4)
forward = [work(s) for s in children]
backward = [work(s) for s in reversed(np.random.SeedSequence(7).spawn(4))][::-1]

print("four worker means:", [round(float(m), 4) for m in forward])
print("same in any order:", forward == backward)
print("different streams:", len(set(forward)) == 4)
```

```
four worker means: [-0.0682, 0.0625, 0.0279, -0.0366]
same in any order: True
different streams: True
```

Each child sequence is derived from the parent in a way that is designed to give independent streams. The result of worker 2 does not depend on whether worker 1 ran first. This is the property that lets you move from a loop to a pool of processes and keep the numbers.

What the documentation promises, and what it does not

The numbers above are stable on one version of NumPy. The NumPy policy is that the streams of `Generator` methods are not guaranteed to be the same between versions. A seed makes a run reproducible on one installation. It does not make it reproducible for ever.

If a published number depends on a stream, write the NumPy version next to the seed. This book does so. NumPy 2.5.4 produced every output.

The function you build

A line fit gives one slope. To say how well the data fix the slope, resample the rows with replacement, fit again, and look at the spread of the slopes. This is the bootstrap. Doing 1000 fits is a loop in Python, and a few lines of broadcasting here.

```
import numpy as np


def bootstrap_slopes(x, y, n_boot, seed):
    """Slope of the least-squares line for `n_boot` resamples of the rows, all at once."""
    g = np.random.default_rng(seed)
    idx = g.integers(0, len(x), (n_boot, len(x)))
    xs, ys = x[idx], y[idx]
    xc = xs - xs.mean(axis=1, keepdims=True)
    yc = ys - ys.mean(axis=1, keepdims=True)
    return (xc * yc).sum(axis=1) / (xc * xc).sum(axis=1)


def fit_line(x, y, n_boot=1000, seed=0):
    """Return (intercept, slope, (low, high)) with a 95% bootstrap interval for the slope."""
    X = np.column_stack([np.ones_like(x), x])
    (b0, b1), *_ = np.linalg.lstsq(X, y, rcond=None)
    low, high = np.percentile(bootstrap_slopes(x, y, n_boot, seed), [2.5, 97.5])
    return float(b0), float(b1), (float(low), float(high))


x = np.linspace(0, 1, 40)
y = 1 + 2 * x + np.random.default_rng(5).normal(0, 0.1, 40)

b0, b1, (lo, hi) = fit_line(x, y, seed=3)
print(f"intercept {b0:.3f}, slope {b1:.3f}, 95% interval for slope ({lo:.3f}, {hi:.3f})")
print("same seed, same interval:     ", fit_line(x, y, seed=3) == fit_line(x, y, seed=3))
print("different seed, same interval:", fit_line(x, y, seed=4)[2] == fit_line(x, y, seed=3)
[2])
print("true slope inside interval:   ", lo <= 2.0 <= hi)
```

```
intercept 1.001, slope 1.934, 95% interval for slope (1.850, 2.014)
same seed, same interval:      True
different seed, same interval: False
true slope inside interval:    True
```

The bootstrap function does three steps from earlier chapters. It draws a `(1000, 40)` array of row numbers. It uses an integer index list, from chapter 4, to build `x[idx]` of shape `(1000, 40)`. It uses `keepdims=True`, from chapter 5, to center each row. No loop appears.

Two checks that the module runs are worth stating. First, each bootstrap slope equals the slope that `lstsq` gives for the same resample, so the closed-form shortcut is right. Second, a bootstrap interval is a claim about coverage, and you can measure the claim. Fit 300 data sets that have a true slope of 2, build the interval for each, and count how often the interval holds 2.

Run this block from the root of the book, so that the import finds the module.

```
import numpy as np

from examples.ch07_linalg_random import fit_line

x = np.linspace(0, 1, 40)
trials, covered = 300, 0
for t in range(trials):
    y = 1 + 2 * x + np.random.default_rng(1000 + t).normal(0, 0.1, 40)
    _, _, (lo, hi) = fit_line(x, y, n_boot=500, seed=t)
    covered += lo <= 2.0 <= hi
print(f"covered {covered} of {trials}: {covered / trials:.3f}")
```

```
covered 289 of 300: 0.963
```

The interval is nominally 95 percent. For 300 trials, the standard error of a proportion near 0.95 is 0.013. A result between 0.92 and 0.98 agrees with the claim. The measured coverage is in that range.

What you have now

You have `fit_line`, which fits a line and reports an interval, and which gives the same numbers for the same seed. You have three rules, each with a measurement behind it. `solve` has a residual at the rounding level when `inv` does not. `lstsq` keeps 7 digits that the normal equations lose at degree 9. A spawned seed sequence gives results that do not depend on the order of the workers.

Chapter 8 puts the whole book to work on one program. It ends with a list of the traps in the order that you meet them.

Chapter 8 — Putting it together

> **Build** — `kmeans`, a clustering routine of 20 lines with no Python loop over points, a loop version to check it, and a best-of-10 wrapper that fixes the one failure that the check finds.
>
> **Theory kept** — a vectorized program is a chain of the parts from earlier chapters, and each part has a cost in time, memory or precision. You test the chain against a slow, plain version of the same program.
>
> **Source** — New. The algorithm is Lloyd's k-means, which is standard. Every output below is from this book's code.
>
> **Run this first** — `uv run python -m examples.ch08_kmeans`, which is `examples/ch08_kmeans.py`.

The earlier chapters each gave one tool. This chapter uses four of them in one program and checks the result. The program is k-means clustering. It is small enough to read on one page. It needs a distance matrix (chapter 5), a group-by sum (chapter 4), a random generator that you pass (chapter 7) and the checks from chapters 2 and 3.

The algorithm in two lines

Start with `k` centroids. Repeat two steps until the labels stop changing. First, give each point the label of its nearest centroid. Second, move each centroid to the mean of the points that have its label.

The first step is a distance matrix and an `argmin`. The second step is a group-by sum divided by a count. Both are in the book already.

The function you build

```
import numpy as np

from examples.ch05_broadcasting import dist_dot_centered      # chapter 5


def kmeans(X, k, seed=0, max_iter=100):
    """Lloyd's algorithm. Returns (centroids, labels, iterations)."""
    g = np.random.default_rng(seed)                           # chapter 7
    C = X[g.choice(len(X), k, replace=False)].copy()          # chapter 3: copy, not a view
    labels = np.full(len(X), -1)
    for it in range(1, max_iter + 1):
        new_labels = dist_dot_centered(X, C).argmin(axis=1)   # chapter 5
        if np.array_equal(new_labels, labels):
            return C, labels, it - 1
        labels = new_labels
        sums = np.zeros_like(C)
        np.add.at(sums, labels, X)                            # chapter 4
        counts = np.bincount(labels, minlength=k)
        keep = counts > 0                                     # an empty cluster keeps its
centroid
        C[keep] = sums[keep] / counts[keep, None]             # chapter 5: broadcast a column
    return C, labels, max_iter


centers = np.array([[0.0, 0.0], [6.0, 0.0], [3.0, 5.0]])
g = np.random.default_rng(1)
X = np.concatenate([c + 0.8 * g.standard_normal((300, 2)) for c in centers])
truth = np.repeat(np.arange(3), 300)

C, labels, iters = kmeans(X, 3, seed=1)
print("iterations:", iters)
print("centroids:")
print(C.round(2))
print("cluster sizes:", np.bincount(labels))
```

```
iterations: 3
centroids:
[[ 5.97 -0.06]
 [-0.07  0.  ]
 [ 3.02  5.02]]
cluster sizes: [300 300 300]
```

The three true centers are `(0, 0)`, `(6, 0)` and `(3, 5)`. The centroids are close to these, in another order. Each cluster has about 300 points. The `for` loop runs over iterations, and there are few of them. No loop runs over points.

The comments name the chapter of each line. The line `C[keep] = sums[keep] / counts[keep, None]` has a trap that chapter 5 described. The count has shape `(k,)` and the sums have shape `(k, d)`. Without `None`, the shapes `(k, d)` and `(k,)` do not align, or they align wrongly when `d` equals `k`.

Check it against the slow version

A fast version that you did not check is a guess. The module has a second function, `kmeans_loop`, with the same algorithm and explicit Python loops, and the same starting centroids.

```
import numpy as np

from examples.ch08_kmeans import kmeans, kmeans_loop, make_blobs

centers = np.array([[0.0, 0.0], [6.0, 0.0], [3.0, 5.0]])
X, truth = make_blobs(300, centers, 0.8, seed=1)

C, labels, it = kmeans(X, 3, seed=1)
Cl, labels_l, it_l = kmeans_loop(X, 3, seed=1)
print("same labels:        ", np.array_equal(labels, labels_l))
print("same centroids:     ", np.allclose(C, Cl))
print("same iteration count:", it == it_l)
print("largest centroid difference:", np.abs(C - Cl).max())
```

```
same labels:         True
same centroids:      True
same iteration count: True
largest centroid difference: 0.0
```

The two versions agree on every label, and the largest centroid difference is 0.0 on this input. On other data the two versions can add in a different order and differ at the level of rounding. This is why the check uses `np.allclose` for the centroids and not `==`. If one label differs, the cause is almost always a tie that rounding broke the other way. In that case, compare the inertia.

The check that found a real problem

Run the same function from 30 different seeds. The data are the same, and only the starting centroids differ. Count how often the run finds the three blobs, which we measure with *purity*: the fraction of points whose cluster has the same majority true label as they do.

```
import numpy as np

from examples.ch08_kmeans import kmeans, kmeans_best, make_blobs, purity, inertia

centers = np.array([[0.0, 0.0], [6.0, 0.0], [3.0, 5.0]])
X, truth = make_blobs(300, centers, 0.8, seed=1)

single = [purity(kmeans(X, 3, seed=s)[1], truth, 3) for s in range(30)]
good = sum(p > 0.95 for p in single)
print(f"single start: {good} of 30 seeds find the blobs")
print("purity of the failures:", sorted({round(float(p), 2) for p in single if p <= 0.95}))

inertias = {}
for s in range(30):
    C, labels, _ = kmeans(X, 3, seed=s)
    inertias[s] = (purity(labels, truth, 3) > 0.95, inertia(X, C, labels))
ok = [v for good, v in inertias.values() if good]
bad = [v for good, v in inertias.values() if not good]
print(f"inertia of the {len(ok)} good runs:    {min(ok):.1f} to {max(ok):.1f}")
print(f"inertia of the {len(bad)} failed runs:  {min(bad):.1f} to {max(bad):.1f}")

C, labels, _ = kmeans(X, 3, seed=0)
print("seed 0: rows are clusters, columns are true blobs")
print(np.array([np.bincount(truth[labels == j], minlength=3) for j in range(3)]))
```

```
single start: 24 of 30 seeds find the blobs
purity of the failures: [0.67]
inertia of the 24 good runs:    1176.7 to 1176.7
inertia of the 6 failed runs:  6215.8 to 6503.7
seed 0: rows are clusters, columns are true blobs
[[  0 157   5]
 [  0 143   0]
 [300   0 295]]
```

In 6 of 30 runs, the algorithm stops at a purity of 0.67. The loop version agrees with the vectorized version on seed 1, so the cause is not a bug in the NumPy code. It is a property of the algorithm. The table for seed 0 shows the shape of the failure. One cluster holds all of blob 0 and nearly all of blob 2, so its centroid sits between them. The other two clusters split blob 1.

No single step improves this result. It is a local minimum.

Each run has an inertia, which is the sum of squared distances from the points to their centroids. The good runs all have the same inertia, and every failed run has a larger one. The inertia ranks the runs without the true labels, and this is why the standard fix works. Run the algorithm from several starts, and keep the run with the lowest inertia. The function `kmeans_best` does this with `SeedSequence.spawn`, as chapter 7 recommends.

```
import numpy as np

from examples.ch08_kmeans import kmeans, kmeans_best, make_blobs, purity

centers = np.array([[0.0, 0.0], [6.0, 0.0], [3.0, 5.0]])
X, truth = make_blobs(300, centers, 0.8, seed=1)

best = [purity(kmeans_best(X, 3, n_init=10, seed=s)[1], truth, 3) for s in range(30)]
print(f"best of 10 starts: {sum(p > 0.95 for p in best)} of 30 seeds find the blobs")
```

```
best of 10 starts: 30 of 30 seeds find the blobs
```

With 10 starts, all 30 seeds find the blobs. If one start fails with probability 0.2, the chance that all 10 fail is 0.2 to the power 10, which is 1e-7. The 30 results agree with that arithmetic.

What the vectorization bought

```
import timeit

import numpy as np

from examples.ch08_kmeans import kmeans, kmeans_loop, make_blobs

centers = np.array([[0.0, 0.0], [6.0, 0.0], [3.0, 5.0]])
Xs, _ = make_blobs(300, centers, 0.8, seed=2)
t_loop = min(timeit.repeat(lambda: kmeans_loop(Xs, 3, seed=1), number=1, repeat=2))
t_vec = min(timeit.repeat(lambda: kmeans(Xs, 3, seed=1), number=1, repeat=5))
assert t_loop / t_vec > 20
print(f"900 points: loop {t_loop * 1e3:.0f} ms, vectorized {t_vec * 1e3:.1f} ms")

Xb, _ = make_blobs(100_000, centers, 0.8, seed=3)
t_big = min(timeit.repeat(lambda: kmeans(Xb, 3, seed=1), number=1, repeat=3))
print(f"300,000 points, vectorized: {t_big * 1e3:.0f} ms")
```

```
# output varies
900 points: loop 29 ms, vectorized 0.8 ms
300,000 points, vectorized: 319 ms
```

On 900 points the vectorized version is about 36 times faster on the build machine, and the assert requires 20. The loop time scales with the number of points. At 300,000 points it is 333 times larger, which is about 10 seconds, and the run does not include it.

This is the usual pattern. The check runs on a small input, and the fast version runs on the real one.

The traps, in the order you meet them

Each row is a mistake that gives a wrong answer and no error. The chapter column says where this book measured it.

Trap	Why it happens	How to catch it	Chapter
An array has a different dtype than you assumed	NumPy picks the dtype from the data	Print `.dtype` at every input and output	2
Integers wrap	The dtype fixes the number of bits	`checked_add`, or a wider dtype	2
A `float32` running sum drifts	Small values are added to a large total	Use `x.sum()`, or `float64`	2
A function changes its argument	A slice is a view	`np.shares_memory`, `root`	3
A small view keeps a large block alive	The view holds a reference to its base	`.copy()`	3
A write is lost	A mask or index list makes a copy	Write with the mask on the left	4
A repeated index is applied once	`a[i] += 1` reads, adds, then writes	`np.add.at`, `np.bincount`	4
A result is `(n, n)` and not `(n,)`	Broadcasting stretches a length-1 axis	Assert the shape of a result	5
The distance matrix is wrong in `float32`	`|a|² + |b|² − 2a·b` subtracts large numbers	Center the data first	5
`np.vectorize` is no faster	It is a Python loop	Use ufuncs and `np.where`	6
`inv(A) @ b` is inaccurate	The inverse amplifies rounding	`solve`, and check the residual	7
A seed gives other numbers on another machine	The stream depends on the NumPy version	Write the version next to the seed	7
An iterative method stops at a poor answer	A local minimum	Several starts, keep the lowest inertia	8

The most useful single habit is the test in this chapter. Write the slow version first. It is short, it is obviously right, and it costs nothing to keep. Then write the fast version, and assert that the two agree on a small input.

What this book left out

This book is short, so it skips topics that have a real use. They are listed here so that you know where the edge is. This book did not measure any of them.

- `np.einsum` writes a sum over named axes in one call, and replaces many `@` and broadcast chains.
- Structured arrays and `np.recarray` hold records with named fields of mixed types.
- Masked arrays hold missing values with a mask and not a `nan`.
- `np.memmap` and `np.load(mmap_mode="r")` read an array from disk without loading it.
- `np.fft` computes Fourier transforms, and `np.linalg` has factorizations beyond those used here.
- Compilers such as Numba, and libraries such as SciPy, cover the loops that chapter 6 could not vectorize.

What you have now

You have a clustering routine of 20 lines that you checked against a slow version, and a wrapper that fixes its one measured failure. You have a table of 13 traps, each with a measurement in an earlier chapter.

The book made one claim, and every chapter tested it: an array program is fast because of how it uses memory, and it is wrong in the places where it hides that use. The shape, the dtype and the strides from chapter 1 explain every row of the table above.

Exercises — Chapter 1

Chapter: The array · module: `examples/ch01_array.py` · solutions: 01

1. [measure] Make `np.zeros((100, 200, 3), dtype=np.float32)`. Report its `strides` and its `nbytes`. Expected: strides `(2400, 12, 4)` and `nbytes` 240000, exact.

2. [break] Chapter 1 says that a transpose is not C-contiguous. Find a 2-D array for which the transpose is still C-contiguous. Explain why the flag stays `True`. Expected: a shape with an axis of length 1, for example `(1, 5)`.

3. [judge] A colleague stores 20 sensor readings per request in a NumPy array, and the service handles thousands of requests each second. Another colleague says that a plain list is better. What does a good answer weigh? (Consider the fixed cost per call from chapter 1, what the readings are used for next, and what a measurement on the real service must show.)

Exercises — Chapter 2

Chapter: Dtypes, and the answers that change silently · module: `examples/ch02_dtypes.py` · solutions: 02

1. [measure] Find the smallest positive integer `n` for which `np.float32(n) + np.float32(1) == np.float32(n)`. Expected: 16777216, exact.

2. [break] The cumulative sum of ten million `float32` values of 0.1 is wrong by 8.8 percent in chapter 2. Change one argument of `np.cumsum`, and nothing else, so that the last value is within 1e-6 (relative) of 1,000,000.015. Expected: `dtype=np.float64`, and a relative error below 1e-6.

3. [judge] A library function adds two `uint8` images. What must it do on overflow: wrap, clip at 255, or raise an error? What does a good answer weigh? (Consider who calls the function, what a wrapped pixel looks like, and what each choice costs in speed and in surprise.)

Exercises — Chapter 3

Chapter: Views and copies · module: `examples/ch03_views.py` · solutions: 03

1. [measure] Make `big = np.zeros((1000, 1000))` and `small = big[:10]`. How many bytes does `small` keep alive through `small.base`? Expected: 8000000, exact.

2. [break] The function `center` in chapter 3 changes the column that the caller passes in. Change one line so that the caller's data stay the same, and show that they do. Expected: the original array equals its copy from before the call.

3. [judge] You write a public function that takes an array and returns a part of it. Does it return a view or a copy? What does a good answer weigh? (Consider memory, the caller's later writes, and what the documentation can promise.)

Exercises — Chapter 4

Chapter: Indexing · module: `examples/ch04_indexing.py` · solutions: 04

1. [measure] Let `idx = np.array([0, 0, 0, 1, 1, 2])`. Add 1 at each index of `np.zeros(3, dtype=int)` in two ways.

- Use `a[idx] += 1`.
- Use `np.add.at`.

Report both results. Expected: `[1 1 1]` and `[3 2 1]`, exact.

2. [break] The `np.where(x > 0, np.log(x), 0)` call in chapter 4 gives a warning and not an error. Wrap the call so that it raises `FloatingPointError`, and say what that shows about `np.where`. Expected: the message `divide by zero encountered in log`.

3. [judge] A team uses `np.add.at`. They say that `np.bincount` is "known to be faster". Chapter 4 measured the two as equal on one version of NumPy. What does a good answer weigh before the team changes the code?

Consider the version, the key range, the profile of their own data, and the cost of the change.

Exercises — Chapter 5

Chapter: Broadcasting · module: `examples/ch05_broadcasting.py` · solutions: 05

1. [measure] Use `tracemalloc` to measure the peak memory of `dist_broadcast` for two sets of 1000 points in 32 dimensions. Expected: 264 MB, within ±10 percent. State the number of values in the temporary array and the result of the arithmetic.

2. [break] Make `dist_dot` return a maximum error larger than 1.0 for `float64` points that lie in a cube of side 1. Change only the position of the cube. Expected: an offset of 1e8 gives an error of about 4. Explain the result with the gap between neighbor `float64` values.

3. [judge] Your service computes distances for 50 points against 50,000 points in 128 dimensions, in `float32`, on a machine with 8 GB. Which of the three versions do you ship? What does a good answer weigh? (Consider the size of the temporary array, the precision, and whether centering is possible when the second set changes at each call.)

Exercises — Chapter 6

Chapter: Ufuncs and vectorization · module: `examples/ch06_ufuncs.py` · solutions: 06

1. [measure] For 100,000 values, measure the ratio of the time of `ma_window` to the time of `ma_cumsum` at window 10 and at window 1000. Expected: the ratio at window 1000 is above 10, and it is larger than the ratio at window 10. Say why the ratio depends on the window.

2. [break] `np.vectorize(lambda v: 0 if v < 1 else v)(np.array([0.5, 2.7]))` must give `[0. 2.7]`. Run it and report the result. Then repair it with one argument. Expected: the result `[0 2]` first, and `[0. 2.7]` after the repair.

3. [judge] A colleague wants to rewrite the exponential moving average of chapter 6 in Numba. What does a good answer weigh? (Consider the measured 3.6 million steps a second, the size of the real data, the cost of a new dependency, and what a profile must show first.)

Exercises — Chapter 7

Chapter: Linear algebra and random numbers · module: `examples/ch07_linalg_random.py` · solutions: 07

1. [measure] Compute `np.linalg.cond` of the 10-by-10 Hilbert matrix. Expected: 1.6e13, within a factor of 2.

2. [break] `np.linalg.solve` gave an error of 1.2e-7 for the 8-by-8 Hilbert system in `float64`. Change only the dtype of the inputs so that the error is larger than 1. Expected: `float32` gives an error near 7. Explain the result with the condition number and the `float32` epsilon.

3. [judge] A team of five runs experiments. Each script has `np.random.seed(0)` at the top. The scripts import shared helper functions that also draw random numbers. What does a good answer weigh before the team changes its practice? (Consider hidden coupling, reproducibility across versions, and the cost of passing a generator through every function.)

Exercises — Chapter 8

Chapter: Putting it together · module: `examples/ch08_kmeans.py` · solutions: 08

1. [measure] Use `make_blobs(300, centers, 0.8, seed=2)` with the centers of chapter 8. Run `kmeans` once from each of the seeds 0 to 29. Count the runs that have a purity of 0.95 or less. Expected: 5, exact.

2. [break] Replace the starting centroids in `kmeans` with the view `X[:k]`, and remove the `.copy()`. Run the function and report whether the input `X` changed. Expected: `X` changed. Explain which chapter predicted it.

3. [judge] Best-of-10 starts costs ten times as much as one start. A colleague says that a smarter initialization such as k-means++ makes it unnecessary. What does a good answer weigh? (Consider the measured failure rate of 6 in 30, the cost per run at your data size, and what you can measure on your data before you decide.)

Solutions — Chapter 1

Exercises: 01 · chapter: The array

1. [measure]

The strides are the number of bytes to jump along each axis. The last axis has 3 values of 4 bytes, so its stride is 4. The middle axis jumps over one row of 3 values, which is 12 bytes. The first axis jumps over 200 rows of 12 bytes, which is 2400 bytes.

```
import numpy as np

a = np.zeros((100, 200, 3), dtype=np.float32)
print(a.strides)
print(a.nbytes)
assert a.strides == (2400, 12, 4)
assert a.nbytes == 100 * 200 * 3 * 4
```

```
(2400, 12, 4)
240000
```

2. [break]

The transpose of a `(1, 5)` array is a `(5, 1)` array. An axis of length 1 is never stepped along, so its stride does not matter, and NumPy sets the contiguous flag from the other axes.

```
import numpy as np

a = np.zeros((1, 5))
print("a   shape", a.shape, "strides", a.strides, "contiguous", a.flags.c_contiguous)
print("a.T shape", a.T.shape, "strides", a.T.strides, "contiguous", a.T.flags.c_contiguous)
assert a.T.flags.c_contiguous
assert np.shares_memory(a, a.T)
```

```
a   shape (1, 5) strides (40, 8) contiguous True
a.T shape (5, 1) strides (8, 40) contiguous True
```

The strides changed from `(40, 8)` to `(8, 40)`, and the array is still contiguous. The flag describes the memory order that a walk follows. A walk over a `(5, 1)` array has only one axis that moves.

3. [judge]

Position. Keep the list, unless a measurement on the real service shows that the array helps.

What a good answer weighs. Twenty values is near the size where the fixed cost of an array call is larger than the work. The chapter measured a loss for three values. At thousands of requests each second, the cost per request matters.

The next step of the readings is the second fact. If the next step is a mean and a threshold, a list is enough. If the next step is a matrix product with a model, the model needs an array, and the conversion cost is paid once. The measurement must use the real request rate and the real next step. A benchmark of 20 values alone is not enough.

Strongest objection. The array gives the same code path as the offline batch job that runs on millions of readings. One code path has fewer bugs. This is an argument about maintenance and not about speed.

What overturns the position. A profile can show that the conversion costs less than 5 percent of the request time. A requirement to share code with the batch job has the same effect.

Solutions — Chapter 2

Exercises: 02 · chapter: Dtypes, and the answers that change silently

1. [measure]

A `float32` has 24 bits of precision. Below 2^{24} every integer is exact and has a gap of at most 1 to its neighbor. At 2^{24} the gap becomes 2, and adding 1 rounds back to the same number.

```
import numpy as np

n = np.arange(1, 2 ** 24 + 2, dtype=np.float32)        # every integer up to 2**24 + 1
stuck = np.flatnonzero(n + np.float32(1) == n)
smallest = int(n[stuck[0]])
print(smallest)
assert smallest == 2 ** 24 == 16777216
```

```
16777216
```

The test runs on every integer from 1 to $2^{24} + 1$, so it finds the smallest `n` and not only a power of 2.

2. [break]

The argument is `dtype=np.float64`. The cumulative sum then keeps its running total in `float64`, which has digits to spare.

```
import numpy as np

x = np.full(10_000_000, 0.1, dtype=np.float32)
exact = float(np.float32(0.1)) * x.size

plain = float(np.cumsum(x)[-1])
wide = float(np.cumsum(x, dtype=np.float64)[-1])
print(f"plain cumsum relative error: {abs(plain - exact) / exact:.2e}")
print(f"float64 cumsum relative error: {abs(wide - exact) / exact:.2e}")
assert abs(plain - exact) / exact > 1e-2
assert abs(wide - exact) / exact < 1e-6
```

```
plain cumsum relative error: 8.79e-02
float64 cumsum relative error: 0.00e+00
```

The input is still `float32`. Only the accumulator changed. The cost is memory: the result of `cumsum` is now 80 MB and not 40 MB.

3. [judge]

Position. Raise an error by default. Offer a `clip=True` option for the caller that wants saturation. Wrapping is almost never what the caller means.

What a good answer weighs. The caller of an image function usually wants a bright pixel to stay bright. That is a clip. A wrapped pixel turns a bright area dark, and the image looks plausible and is wrong. An error stops the program at the first bad input. The cost is one extra comparison pass.

A clip hides that the input was out of range, and that fact can be the information that the caller needs. Wrapping is correct for hash functions and modular counters. These are not images.

Strongest objection. In a video pipeline that runs at 60 frames a second, an error stops the stream on one bad frame. For this caller the silent clip is the safe choice. The answer is the option: the default is the error, and the pipeline sets `clip=True`.

What overturns the position. A measurement can show that the check costs more than the budget of the caller. An example is 10 percent of the frame time. Then the default must change, and the first line of the documentation must say so.

Solutions — Chapter 3

Exercises: 03 · chapter: Views and copies

1. [measure]

```
import numpy as np

big = np.zeros((1000, 1000))
small = big[:10]
print("small owns:", small.nbytes, "bytes")
print("small keeps:", small.base.nbytes, "bytes alive")
assert small.base.nbytes == 8_000_000
assert small.nbytes == 80_000
```

```
small owns: 80000 bytes
small keeps: 8000000 bytes alive
```

The slice has 10 rows of 1000 values of 8 bytes, which is 80,000 bytes. The base has 1000 rows, which is 8,000,000 bytes. The ratio is 100.

2. [break]

Replace `x -= x.mean()` with `x = x - x.mean()`. The second form makes a new array and binds the name `x` to it. The array of the caller is not touched.

```
import numpy as np


def center(x):
    x = x - x.mean()
    return x


data = np.array([[1.0, 10.0], [3.0, 20.0], [5.0, 30.0]])
before = data.copy()
centered = center(data[:, 0])
print("data unchanged:", np.array_equal(data, before))
print("result:", centered)
assert np.array_equal(data, before)
```

```
data unchanged: True
result: [-2.  0.  2.]
```

3. [judge]

Position. Return a view when the function is a selection that the caller keeps in step with the original. Return a copy when the function computes something. Write the choice in the first line of the docstring.

What a good answer weighs. A view costs nothing and keeps the base alive, which can be a large block. A copy costs memory and time and breaks the link. A later write by the caller does not change the original.

The danger of a view is a write that the caller does not expect. The danger of a copy is a write that the caller expects and does not get. NumPy returns views for slices and copies for index lists, so a caller who knows NumPy expects the same pattern.

Strongest objection. A function that returns a view or a copy depending on the input, like `reshape`, is a trap. It is better to always copy. This is correct for a public function. The cost is memory on large arrays, and the answer is to document the exception where the cost is measured.

What overturns the position. A user report of a lost write or an unexpected change shows that callers do not read the docstring. Then return copies, and add a separate function that returns a view.

Solutions — Chapter 4

Exercises: 04 · chapter: Indexing

1. [measure]

```
import numpy as np

idx = np.array([0, 0, 0, 1, 1, 2])

a = np.zeros(3, dtype=int)
a[idx] += 1
b = np.zeros(3, dtype=int)
np.add.at(b, idx, 1)
print("a[idx] += 1:", a)
print("np.add.at  :", b)
assert a.tolist() == [1, 1, 1]
assert b.tolist() == [3, 2, 1]
```

```
a[idx] += 1: [1 1 1]
np.add.at  : [3 2 1]
```

Index 0 appears 3 times and index 1 appears 2 times. The statement `a[idx] += 1` reads `a[idx]` once, so each position gets only one update.

2. [break]

`np.errstate(all="raise")` turns every floating-point warning into an exception in its block.

```
import numpy as np

x = np.array([1.0, 0.0])
with np.errstate(all="raise"):
    try:
        np.where(x > 0, np.log(x), 0)
    except FloatingPointError as e:
        print(type(e).__name__, "-", e)
```

```
FloatingPointError - divide by zero encountered in log
```

The error comes from `np.log(x)`, which `np.where` receives as an argument. The condition `x > 0` is false for the second element, and it does not stop the log from running. This shows that `np.where` selects among values that are already computed.

3. [judge]

Position. Do not change the code until a profile of the real data shows that this call is a large part of the run time.

What a good answer weighs. The advice is old. It was true for NumPy 1.x. It is not true on 2.5.4 for the input of chapter 4, where the two are within 25 percent. The version of the team is the first fact to check.

The key range is the second fact. The function `np.bincount` needs non-negative integers and makes an array as long as the largest key. That is wasteful for sparse keys. The third fact is how much of the run time the call takes. A change that saves 20 percent of 2 percent of the run is not worth a review.

Strongest objection. The function `np.bincount` states the intent more directly, and it reads better. This is a fair reason to change. It is a reason about readability, and it needs no speed claim.

What overturns the position. A profile that shows the call above 20 percent of the run time overturns it. A measurement on the team's own data and version that shows `bincount` is faster by a useful margin has the same effect.

Solutions — Chapter 5

Exercises: 05 · chapter: Broadcasting

1. [measure]

The temporary array has `n * m * d` values: 1000 × 1000 × 32 = 32,000,000. At 8 bytes each that is 256 MB. The result adds 1000 × 1000 × 8 bytes, which is 8 MB. The sum is 264 MB.

```
import tracemalloc

import numpy as np

from examples.ch05_broadcasting import dist_broadcast

A = np.random.default_rng(0).random((1000, 32))
tracemalloc.start()
dist_broadcast(A, A)
_, peak = tracemalloc.get_traced_memory()
tracemalloc.stop()
print("values in the temporary array:", 1000 * 1000 * 32)
print("expected MB:", (1000 * 1000 * 32 + 1000 * 1000) * 8 / 1e6)
print("measured peak MB:", round(peak / 1e6))
assert abs(peak / 1e6 - 264) < 26
```

```
values in the temporary array: 32000000
expected MB: 264.0
measured peak MB: 264
```

2. [break]

At an offset of 1e8, the squared norms are about 3e16. The gap between neighbor `float64` values at 3e16 is 4. Each term of the subtraction `|a|² + |b|² − 2a·b` has a rounding error of about one gap. The squared distance is wrong by a few gaps, about 16. The square root of 16 is 4.

```
import numpy as np

from examples.ch05_broadcasting import dist_dot, dist_loop

for offset in (1e4, 1e6, 1e8):
    P = np.random.default_rng(0).random((30, 3)) + offset
    err = np.abs(dist_dot(P, P) - dist_loop(P, P)).max()
    print(f"offset {offset:8.0e}  max error {err:.2e}")
assert err > 1.0
print("gap at 3e16:", np.spacing(3e16))
```

```
offset    1e+04  max error 3.45e-04
offset    1e+06  max error 3.12e-02
offset    1e+08  max error 4.00e+00
gap at 3e16: 4.0
```

The points lie in a cube of side 1, so the true distances are at most 1.7. An error of 4 is larger than every true distance. Centering removes the problem, because it moves the cube to the origin before the subtraction.

3. [judge]

Position. Ship the dot version with centering. Process the 50,000 points in blocks. Test it against the broadcast version on a sample.

What a good answer weighs. The broadcast version needs 50 × 50,000 × 128 × 4 bytes, which is 1.28 GB in `float32`. The machine has 8 GB. One request fits, and a few at the same time fail. The dot version needs about the size of the result, which is 10 MB.

Precision is the open question. Centering with the mean of the 50,000 fixed points works if they do not change between calls. If both sets change, the mean of the 50 query points is a cheaper way to center. A fixed reference vector also works.

Strongest objection. The dot version in `float32` loses digits even after centering, if the points are spread over a large range. The answer is to measure the error that the service cares about. For example, measure whether the nearest neighbor is the same one. Do not measure the distance error.

What overturns the position. Compare with the broadcast version in `float64` on real data. Suppose the nearest neighbor changes for more than a small fraction of the queries. Then use `float64` for the cross term, or compute the final candidates exactly.

Solutions — Chapter 6

Exercises: 06 · chapter: Ufuncs and vectorization

1. [measure]

```
import timeit

import numpy as np

from examples.ch06_ufuncs import ma_cumsum, ma_window

x = np.random.default_rng(0).random(100_000)
ratio = {}
for w in (10, 1000):
    t_window = min(timeit.repeat(lambda: ma_window(x, w), number=1, repeat=3))
    t_cumsum = min(timeit.repeat(lambda: ma_cumsum(x, w), number=1, repeat=3))
    ratio[w] = t_window / t_cumsum
assert ratio[1000] > 10
assert ratio[1000] > ratio[10]
print("window time grows with w:", ratio[1000] > ratio[10])
print(f"ratio at w=10: {ratio[10]:.0f}, at w=1000: {ratio[1000]:.0f}")
```

```
# output varies
window time grows with w: True
ratio at w=10: 5, at w=1000: 51
```

The window version does `w` additions for each output value. The cumulative-sum version does two, whatever the size of `w`. The ratio therefore grows in proportion to `w`, and the numbers above are of that order: 5 at `w = 10` and 51 at `w = 1000`.

2. [break]

`np.vectorize` finds the output type by calling the function on the first element. The first call returns the integer 0, so the output type is `int`, and the value 2.7 is cut to 2. NumPy gives no warning.

```
import numpy as np

f = lambda v: 0 if v < 1 else v
broken = np.vectorize(f)(np.array([0.5, 2.7]))
fixed = np.vectorize(f, otypes=[float])(np.array([0.5, 2.7]))
print("broken:", broken)
print("fixed: ", fixed)
assert broken.tolist() == [0, 2]
assert fixed.tolist() == [0.0, 2.7]
```

```
broken: [0 2]
fixed:  [0.  2.7]
```

The repair is `otypes=[float]`. This is a second reason, after speed, to prefer `np.where` over `np.vectorize`: `np.where(v < 1, 0.0, v)` has no output type to guess.

3. [judge]

Position. Not yet. First measure the run time on the real data.

What a good answer weighs. The loop ran at 3.6 million steps a second. A series of one million values takes 0.3 seconds. A series of 100,000 takes 0.03 seconds. If the job runs once an hour, this is not a problem. Numba adds a dependency, a compile step at startup and a second place for bugs.

If a request has 10,000 series of 100,000 values each, the same arithmetic gives 5 minutes. Then the speed matters. The size of the real data and the time budget decide. The chapter measured only the speed of the loop.

Strongest objection. A library routine for linear filters applies this recurrence in compiled code, with no new compiler. That is a better first step than Numba. This book did not measure it. A good answer says so and measures it before the choice.

What overturns the position. A profile of the real job can show the loop above the time budget. A measurement can then show that the library routine does not meet the budget either. Then Numba is the next step.

Solutions — Chapter 7

Exercises: 07 · chapter: Linear algebra and random numbers

1. [measure]

```
import numpy as np

from examples.ch07_linalg_random import hilbert

c = np.linalg.cond(hilbert(10))
print(f"cond(H10) = {c:.2e}")
assert 0.8e13 < c < 3.2e13
```

```
cond(H10) = 1.60e+13
```

The textbook value is 1.6e13. With a rounding unit of 1.1e-16, the rule of chapter 7 predicts an error of about 2e-3 for `solve` on this system.

2. [break]

The `float32` epsilon is 1.2e-7. The condition number of the 8-by-8 Hilbert matrix is 1.5e10. The product is 1.8e3, which is more than the size of the answer. No digit is guaranteed, and the measured error is of order 1.

```
import numpy as np

from examples.ch07_linalg_random import hilbert

H = hilbert(8)
b = H @ np.ones(8)
e64 = np.abs(np.linalg.solve(H, b) - 1).max()
e32 = np.abs(np.linalg.solve(H.astype(np.float32), b.astype(np.float32)) - 1).max()
print(f"float64 error: {e64:.1e}")
print(f"float32 error: {e32:.1f}")
assert e64 < 1e-5
assert e32 > 1.0
print("cond * eps32 =", f"{np.linalg.cond(H) * np.finfo(np.float32).eps:.1e}")
```

```
float64 error: 1.2e-07
float32 error: 6.8
cond * eps32 = 1.8e+03
```

The rule of thumb gives an upper bound and not a prediction. The measured error is 6.8, which is below the bound of 1.8e3 and far above 1.

3. [judge]

Position. Stop using the global seed. Create one generator in the main script. Pass it to every function that draws numbers.

What a good answer weighs. The shared helpers draw from the same global state. A change in one helper changes the numbers in every script that calls it, and nobody sees the link. A passed generator makes the dependency visible in the signature.

The cost is one argument on each function and one decision about who owns the seed. For parallel work, add `SeedSequence.spawn`. No method of seeding promises the same stream across NumPy versions, so each result must also record the version.

Strongest objection. Five scripts already work. A change of every signature touches all of them for no visible gain. This is true for the code that exists. The practical answer is to apply the rule to new code and to the helpers that you change anyway.

What overturns the position. A search of the helpers for `np.random` calls takes minutes. If no helper draws random numbers, the global seed has no hidden coupling, and the change gives little.

Solutions — Chapter 8

Exercises: 08 · chapter: Putting it together

1. [measure]

```
import numpy as np

from examples.ch08_kmeans import kmeans, make_blobs, purity

centers = np.array([[0.0, 0.0], [6.0, 0.0], [3.0, 5.0]])
X, truth = make_blobs(300, centers, 0.8, seed=2)
failures = [s for s in range(30) if purity(kmeans(X, 3, seed=s)[1], truth, 3) <= 0.95]
print("failed seeds:", failures)
print("count:", len(failures))
assert len(failures) == 5
```

```
failed seeds: [0, 6, 13, 15, 19]
count: 5
```

The data of seed 2 give 5 failures in 30. The data of seed 1 in chapter 8 give 6. The failure rate depends on the data, and this is why the chapter reports a count and not a probability.

2. [break]

A basic slice is a view. The first `kmeans` step then writes the new centroids into `C`, which is the first `k` rows of `X`. The input changes, and the three rows that changed are the first three rows of `X`.

```
import numpy as np

from examples.ch05_broadcasting import dist_dot_centered
from examples.ch08_kmeans import make_blobs


def kmeans_view(X, k, max_iter=100):
    C = X[:k]                                   # a view: the .copy() is gone
    labels = np.full(len(X), -1)
    for _ in range(max_iter):
        new_labels = dist_dot_centered(X, C).argmin(axis=1)
        if np.array_equal(new_labels, labels):
            break
        labels = new_labels
        sums = np.zeros_like(C)
        np.add.at(sums, labels, X)
        counts = np.bincount(labels, minlength=k)
        keep = counts > 0
        C[keep] = sums[keep] / counts[keep, None]
    return C, labels


centers = np.array([[0.0, 0.0], [6.0, 0.0], [3.0, 5.0]])
X, _ = make_blobs(300, centers, 0.8, seed=1)
before = X.copy()
C, labels = kmeans_view(X, 3)
print("C shares memory with X:", np.shares_memory(C, X))
print("X unchanged:", np.array_equal(X, before))
print("rows that changed:", int((X != before).any(axis=1).sum()))
assert not np.array_equal(X, before)
```

```
C shares memory with X: True
X unchanged: False
rows that changed: 3
```

Chapter 3 predicted this: the rule is that basic slicing makes a view, and a write through a view changes the original. The original `kmeans` uses `X[g.choice(...)]`, an index list, which always makes a copy, so the `.copy()` there is for safety and not for need. The exercise shows what the safety protects against if a later edit changes the index to a slice.

3. [judge]

Position. Keep best-of-10 until you measure k-means++ on your own data. Then decide with the failure rate and the cost.

What a good answer weighs. On the blobs of chapter 8, 6 of 30 single starts fail, and 10 starts remove the failures. The cost is a factor of 10 in time. A smarter initialization spreads the starting centroids and reduces failures. It also costs a pass over the data. Whether it removes the need for restarts depends on the data and on `k`. This book did not measure it.

At 900 points a run takes under a millisecond, so ten starts is cheap. At 300,000 points a start takes 0.3 seconds. Three seconds for ten starts is small for a batch job and large for a request.

Strongest objection. Restarts treat a property of the initialization as a statistical problem. A better method removes it at the source. This is correct in principle. It is an argument to implement the better initialization and measure it. It is not a reason to remove the restarts before the measurement.

What overturns the position. Measure the better initialization on the real data. Suppose it fails in fewer than 1 run in 100 with a single start. Suppose also that its cost is below that of two plain starts. Then one start is enough.